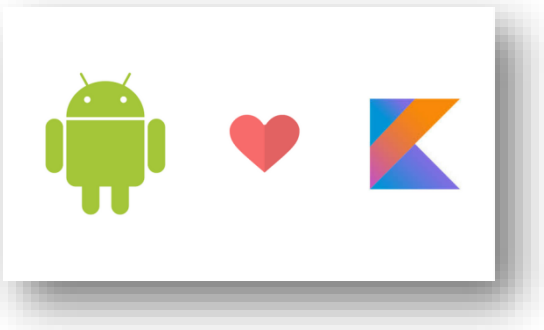
Navigation



Praktikum 6

Tujuan:

Mahasiswa mengenal dan mengimplementasikan:

- Navigation Component
- AppBar, Option Menu, SearchBar dan SearchView
- Navigation Drawer
- Bottom Navigation
- Tab Layout dan View Pager

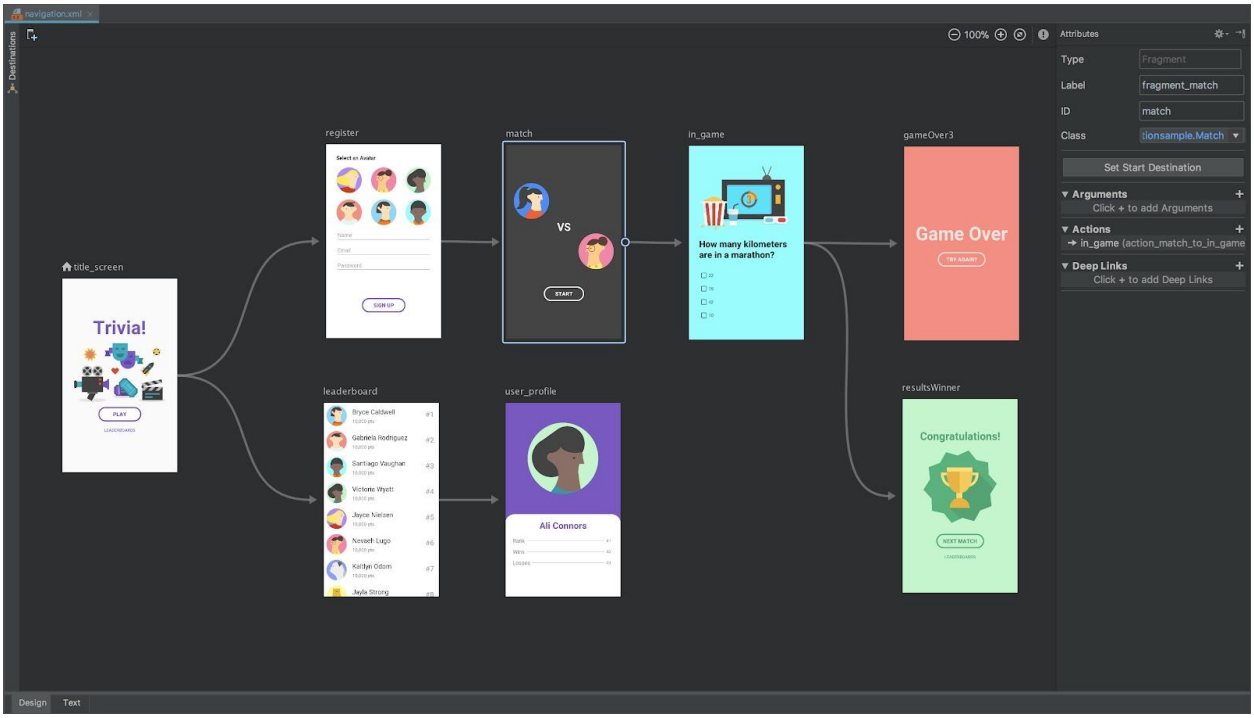
Hasil Praktikum:

- Contoh program praktikum.

Navigation Component

Navigation Component adalah sebuah komponen baru yang termasuk di dalam paket Android Jetpack. Komponen ini memiliki fungsi utama yaitu sebagai navigasi antar Activity maupun Fragment dalam sebuah aplikasi. Bedanya, Navigation Component membuatnya jadi jauh lebih mudah. Jika sebelumnya Anda telah mengenal Intent untuk bernavigasi antar Activity dan FragmentManager untuk bernavigasi antar fragment, sekarang Navigation Component bisa menangani itu semua hanya dalam “beberapa baris kode saja”, termasuk juga dalam hal pengiriman data, bahkan lebih dari itu. Mantap!

Berikut ini adalah contoh aplikasi yang menerapkan Navigation Component:



Kita bisa melakukan navigasi hanya dengan drag and drop di halaman ini. Sebagai catatan, berdasarkan dokumentasi dari Google, Anda harus menggunakan Android Studio versi 3.3 ke atas jika ingin menggunakan fitur ini.

Manfaat Navigation

Berikut ini manfaat dari penggunaan Navigation:

1. Anda bisa melihat alur navigasi dengan lebih mudah dengan adanya visualisasi dari Android Studio
2. Menangani perpindahan Fragment dengan cepat.
3. Menangani aksi navigasi up dan back dengan tepat secara default.
4. Mengatur animasi saat transisi tanpa ribet.
5. Membantu proses saat ingin menerapkan DeepLink.
6. Terdapat NavigationUI, seperti navigation drawer dan bottom navigation.
7. Safe Args, untuk mengirim data tanpa takut adanya error karena data null.

Prinsip Navigation

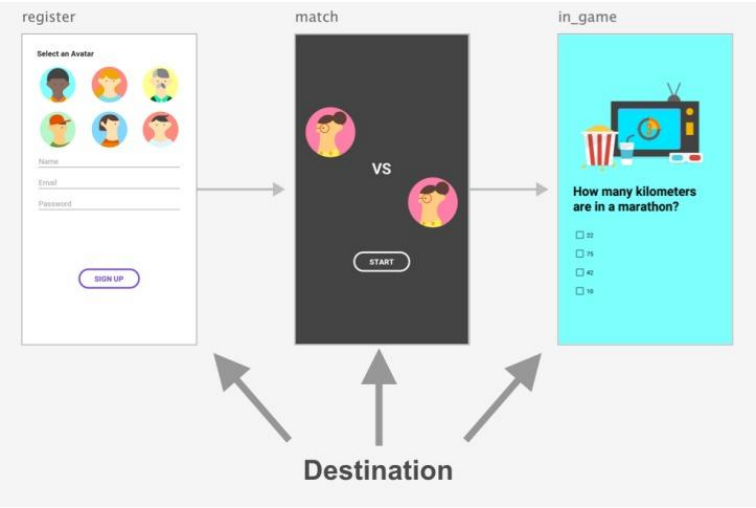
Navigation merupakan aspek penting dalam membangun pengalaman pengguna (user experience) yang baik. Nah, untuk membangun UX yang nyaman itu, Anda perlu mengetahui prinsip-prinsip dasar navigasi supaya aplikasi tetap konsisten dan sesuai harapan pengguna. Berikut ini 5 prinsip Navigation menurut dokumentasi Android.

1. Setiap aplikasi harus memiliki start destination yang tetap. Start destination merupakan halaman awal ketika aplikasi dibuka sehingga selalu menampilkan halaman yang sama saat dijalankan pertama kali.
2. Stack harus menyimpan state saat kembali lagi dari halaman selanjutnya.
3. Tombol Up (yang ada di Top App Bar) dan tombol Back (yang ada di bawah) harus sesuai saat navigasi.
4. Ketika start destination, tombol Up tidak seharusnya menyebabkan keluar dari aplikasi. Sebaliknya Back Button/gesture harus ada untuk bisa keluar dari aplikasi.
5. Deep Link seharusnya memiliki alur backstack yang sama dengan navigasi manual.

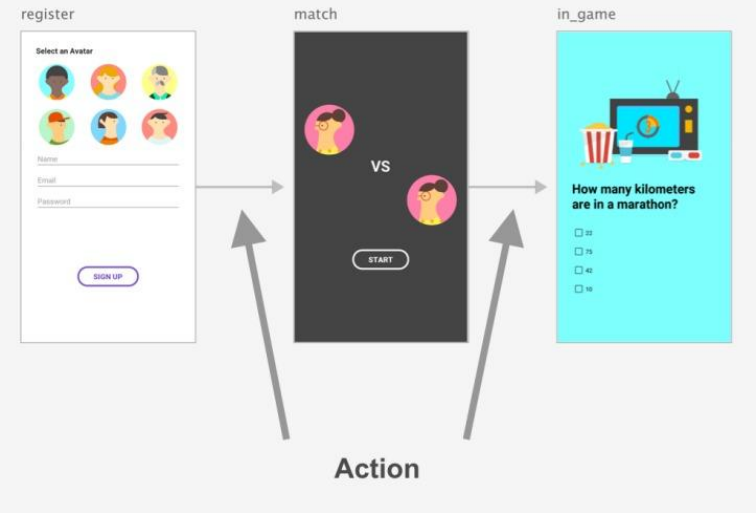
Bagian-Bagian Navigation

Ada 3 bagian dari Navigation Component yang harus Anda ketahui:

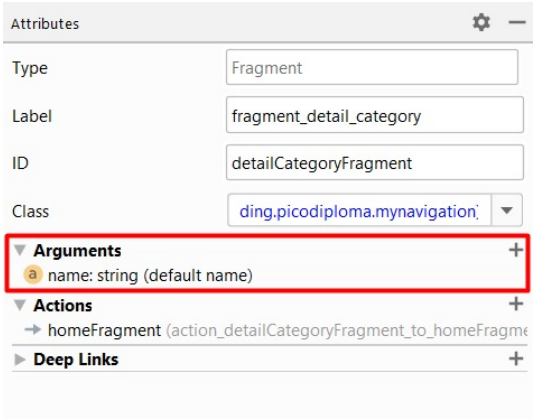
1. Navigation Graph, sebuah kelas resource XML yang letaknya ada di folder res > navigation. Berkas ini berisi semua data navigasi, yaitu seperti:
   - Destination berupa Fragment atau Activity yang saling terhubung.



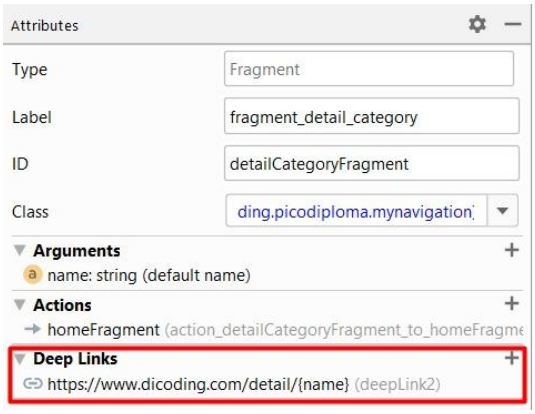
   - Action berupa navigasi dengan destination (tujuan) ke Fragment mana. Disini kita juga bisa mengatur animasi (menerapkan animasi dari folder anim), pop behavior (aksi ketika tombol back) dan juga launch option.



- Argument yang digunakan untuk mendefenisikan data yang akan dikirim antar Fragment.



- Deeplink untuk bisa membuka suatu fragment dengan menggunakan sebuat URL.



2. NavHostFragment adalah layout XML dasar yang berfungsi sebagai host (dasar) dari semua halaman yang lain, biasanya terletak di Activity dengan layout yang berisi fragment. Anda cukup menambahkan tiga baris kode ini supaya fragment tersebut menjadi NavHostFragment

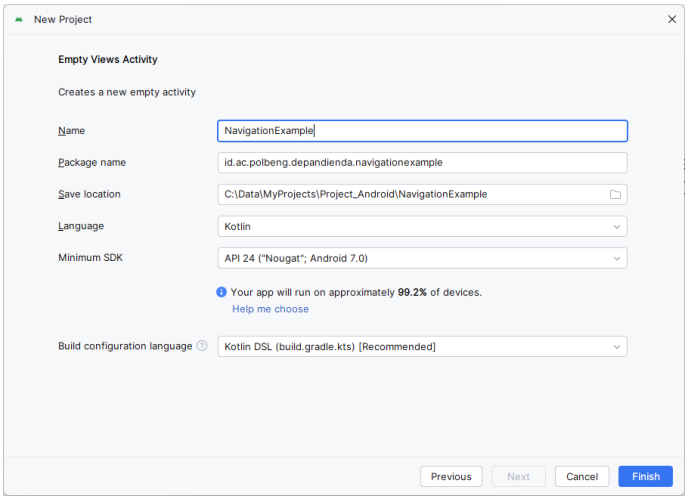
```
<fragment
android:id="@+id/container"
android:name="androidx.navigation.fragment.NavHostFragment"
```

```
android:layout_width="match_parent"
android:layout_height="match_parent"
app:defaultNavHost="true"
app:navGraph="@navigation/main_navigation" />
```

3. NavController adalah kelas yang akan dipakai untuk melakukan eksekusi navigasi yang sudah dibuat.

Latihan Mengimplementasikan Navigation Component

1. Buatlah project baru dengan nama “NavigationExample”.



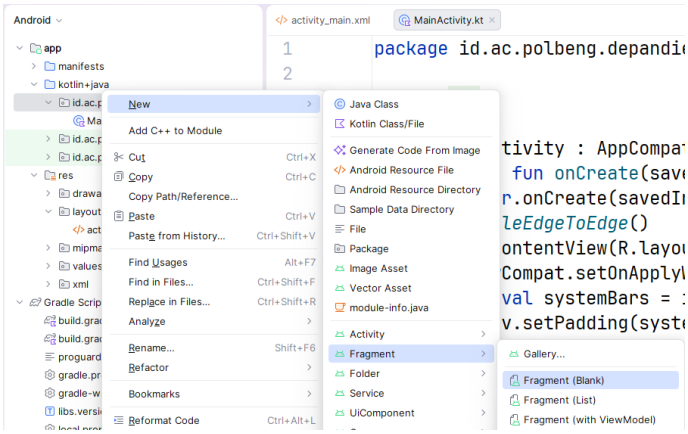
2. Jangan lupa untuk mengaktifkan view-binding dengan menambahkan kode berikut pada build.gradle di level module build.gradle(Module: app), dan tambahkan juga dependency atau library navigation.

```
android {
    ...
    buildFeatures {
        viewBinding = true
    }
}
dependencies {
    ...
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    implementation("androidx.navigation:navigation-fragment-ktx:2.9.7")
    implementation("androidx.navigation:navigation-ui-ktx:2.9.7")
}
```

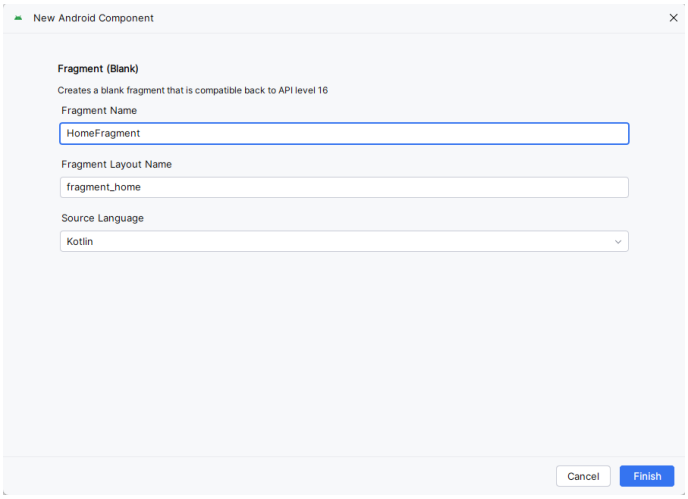
3. Selanjutnya editlah file strings.xml pada folder values, tambahkan beberapa variabel dan nilai string sebagai berikut:

```xml
<resources>
    <string name="app_name">NavigationExample</string>
    <string name="hello_home_fragment">Hello Ini Home Fragment</string>
    <string name="to_profile">Ke Activity Profile Activity</string>
    <string name="to_category">Ke fragment Category</string>
    <string name="this_profile">Ini activity Profile</string>
    <string name="this_category">Ini fragment Category</string>
    <string name="category_lifestyle">Ke fragment Lifestyle</string>
    <string name="category_sport">Ke fragment Sport</string>
    <string name="category_name">Category Name</string>
    <string name="category_description">Category Description</string>
    <string name="to_home">Ke fragment Home</string>
</resources>
```

4. Selanjutnya buat sebuah fragment yang berfungsi sebagai startDestination (tujuan awal) dari Navigation Graph nanti. Caranya : klik kanan pada package utama pada proyek aplikasi Anda → New → Fragment → Fragment (Blank).



Setelah tampil dialog untuk fragment, isikan HomeFragment pada kolom Fragment Name. Klik Finish untuk melanjutkan penciptaan fragment.



Kemudian ubah layout di dalam fragment_home menjadi LinearLayout dan tambahkan dua tombol seperti berikut:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".HomeFragment">

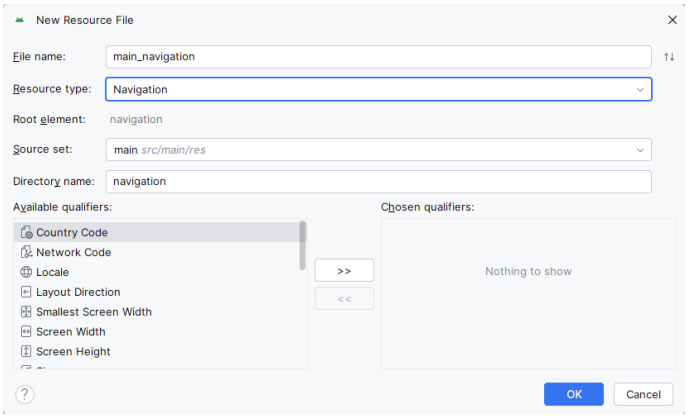
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:text="@string/hello_home_fragment" />

    <Button
        android:id="@+id/btn_category"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginLeft="16dp"
        android:layout_marginRight="16dp"
        android:text="@string/to_category" />

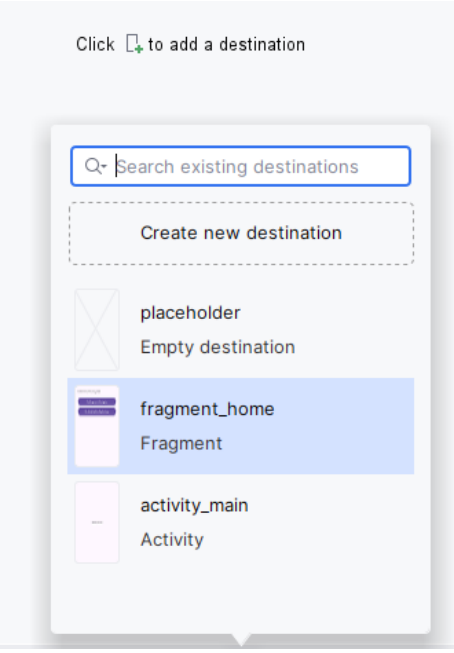
    <Button
        android:id="@+id/btn_profile"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginLeft="16dp"
        android:layout_marginRight="16dp"
        android:text="@string/to_profile" />

</LinearLayout>
```

5. Selanjutnya Anda akan membuat navigation dengan cara klik kanan di folder res → New → Android resource file. Beri nama main_navigation dan pada Resource Type pilih Navigation, setelah selesai Klik OK.



Di dalam main_navigation, pada bagian Design, mulai dengan menambahkan startDestination atau fragment awal yang akan ditampilkan, caranya yaitu dengan klik icon Add destination → pilih fragment_home.



Kemudian beralihlah pada tab Code di tab bagian kanan atas. Maka bagian code di main_navigation menjadi seperti berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main_navigation"
    app:startDestination="@id/homeFragment">
    <fragment
        android:id="@+id/homeFragment"
android:name="id.ac.polbeng.depandienda.navigationexample.HomeFragment"
        android:label="fragment_home"
        tools:layout="@layout/fragment_home" />
</navigation>
```

6. Setelah selesai membuat navigation, lanjut membuat NavHostFragment di layout activity_main.xml dengan cara menambahkan kode berikut:

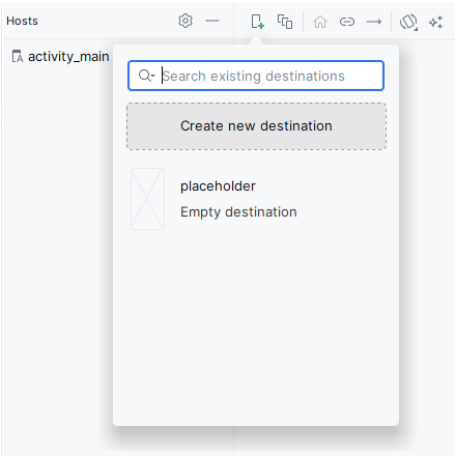
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    tools:context=".MainActivity">

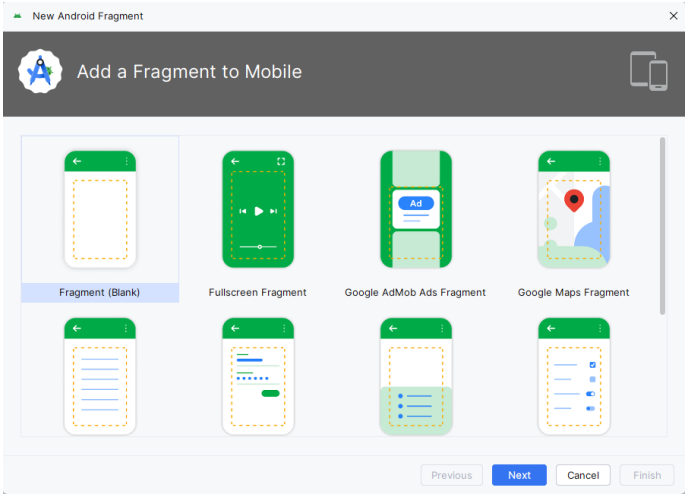
    <fragment
        android:id="@+id/container"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/main_navigation" />

</RelativeLayout>
```

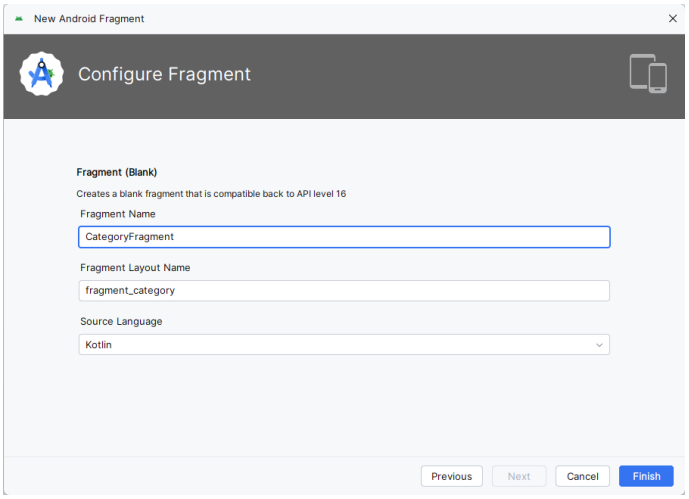
7. Pengaturan dasar navigation sudah selesai, kini tambahkanlah fragment baru dan navigasi di antara keduanya dalam main_navigation pada tab Design. Kemudian klik icon Add Destination dan pilih Create new destination.



Maka akan muncul popup seperti saat membuat fragment baru.



Pilih Fragment(Blank) lalu berilah nama dan klik Finish.



Kemudian ubah layout di dalam fragment_category menjadi seperti ini:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".CategoryFragment">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp"
        android:text="@string/this_category" />

</LinearLayout>
```

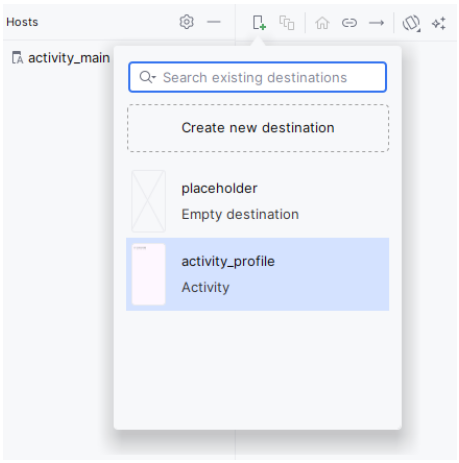
8. Setelah menambahkan fragment, sekarang buat activity baru dengan cara klik kanan pada nama package → New → Activity → Empty Views Activity. Beri nama ProfileActivity dan klik Finish. Kemudian ubah layout pada activity_profile menjadi seperti ini:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/profile"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".ProfileActivity">

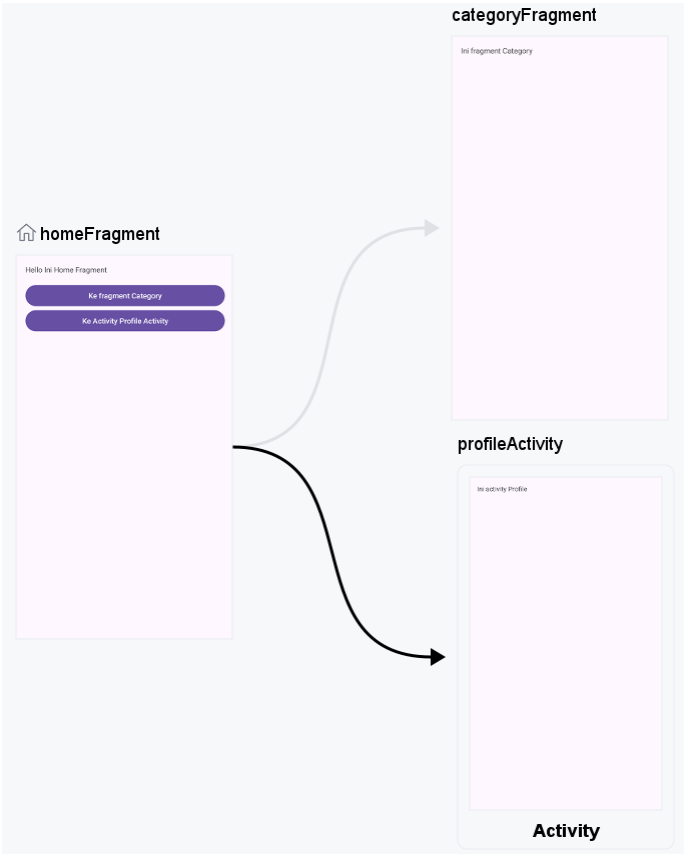
    <TextView
```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/this_profile" />
</RelativeLayout>
```

Kemudian tambahkan ProfileActivity ke dalam main_navigation dengan cara klik AddDestination dan pilih activity_profile.



9. Sekarang saatnya membuat action dari homeFragment ke categoryFragment dengan cara klik lingkaran bulat pada homeFragment kemudian tarik garis ke categoryFragment. lalu sekali lagi tarik garis ke ProfileActivity.



Sehingga pada bagian Code/Text akan tertambah kode berikut:

```xml
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main_navigation"
    app:startDestination="@id/homeFragment">
    <fragment
        android:id="@+id/homeFragment"

android:name="id.ac.polbeng.depandienda.navigationexample.HomeFragment"
        android:label="fragment_home"
        tools:layout="@layout/fragment_home" >
        <action
            android:id="@+id/action_homeFragment_to_categoryFragment"
            app:destination="@id/categoryFragment" />
        <action
            android:id="@+id/action_homeFragment_to_profileActivity"
            app:destination="@id/profileActivity" />
    </fragment>
    <fragment
        android:id="@+id/categoryFragment"

android:name="id.ac.polbeng.depandienda.navigationexample.CategoryFragment"
        android:label="fragment_category"
        tools:layout="@layout/fragment_category" />
    <activity
        android:id="@+id/profileActivity"

android:name="id.ac.polbeng.depandienda.navigationexample.ProfileActivity"
        android:label="activity_profile"
        tools:layout="@layout/activity_profile" />
</navigation>
```

10. Setelah selesai, kemudian panggil action yang sudah Anda buat dengan menggunakan NavController di dalam HomeFragment.

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.navigation.Navigation
import androidx.navigation.findNavController
import
id.ac.polbeng.depandienda.navigationexample.databinding.FragmentHomeBindin
g
```

```
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
                              savedInstanceState: Bundle?): View {
        // Inflate the layout for this fragment
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        val view = binding.root
        return view
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        binding.btnCategory.setOnClickListener(

Navigation.createNavigateOnClickListener(R.id.action_homeFragment_to_categ
oryFragment)
        )
        binding.btnProfile.setOnClickListener { view ->

view.findNavController().navigate(R.id.action_homeFragment_to_profileActiv
ity)
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        _binding = null
    }
}
```

11. Ubahlah kode pada ProfileActivity.kt menjadi seperti berikut:

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import
id.ac.polbeng.depandienda.navigationexample.databinding.ActivityProfileBinding

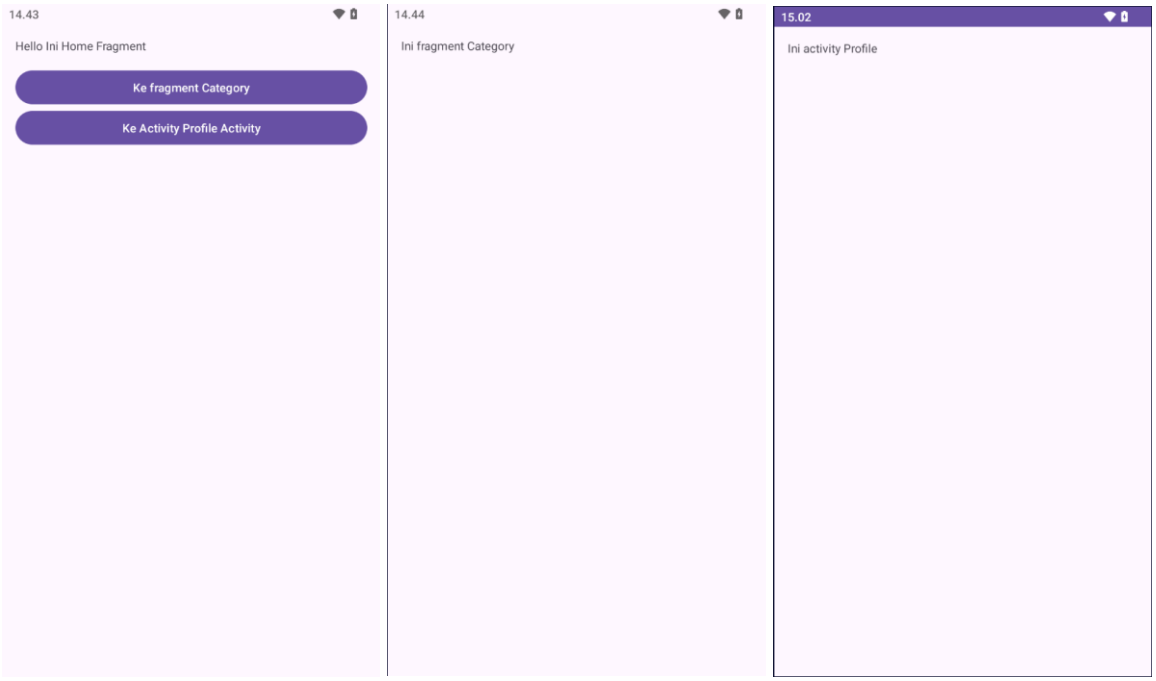
class ProfileActivity : AppCompatActivity() {

    private lateinit var binding: ActivityProfileBinding

    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
        binding = ActivityProfileBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }
}
```

Kemudian coba jalankan aplikasi, maka ia akan menjadi seperti ini:



Mengirim Data dengan Bundle

12. Tambahkan button baru di fragment_category seperti berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".CategoryFragment">

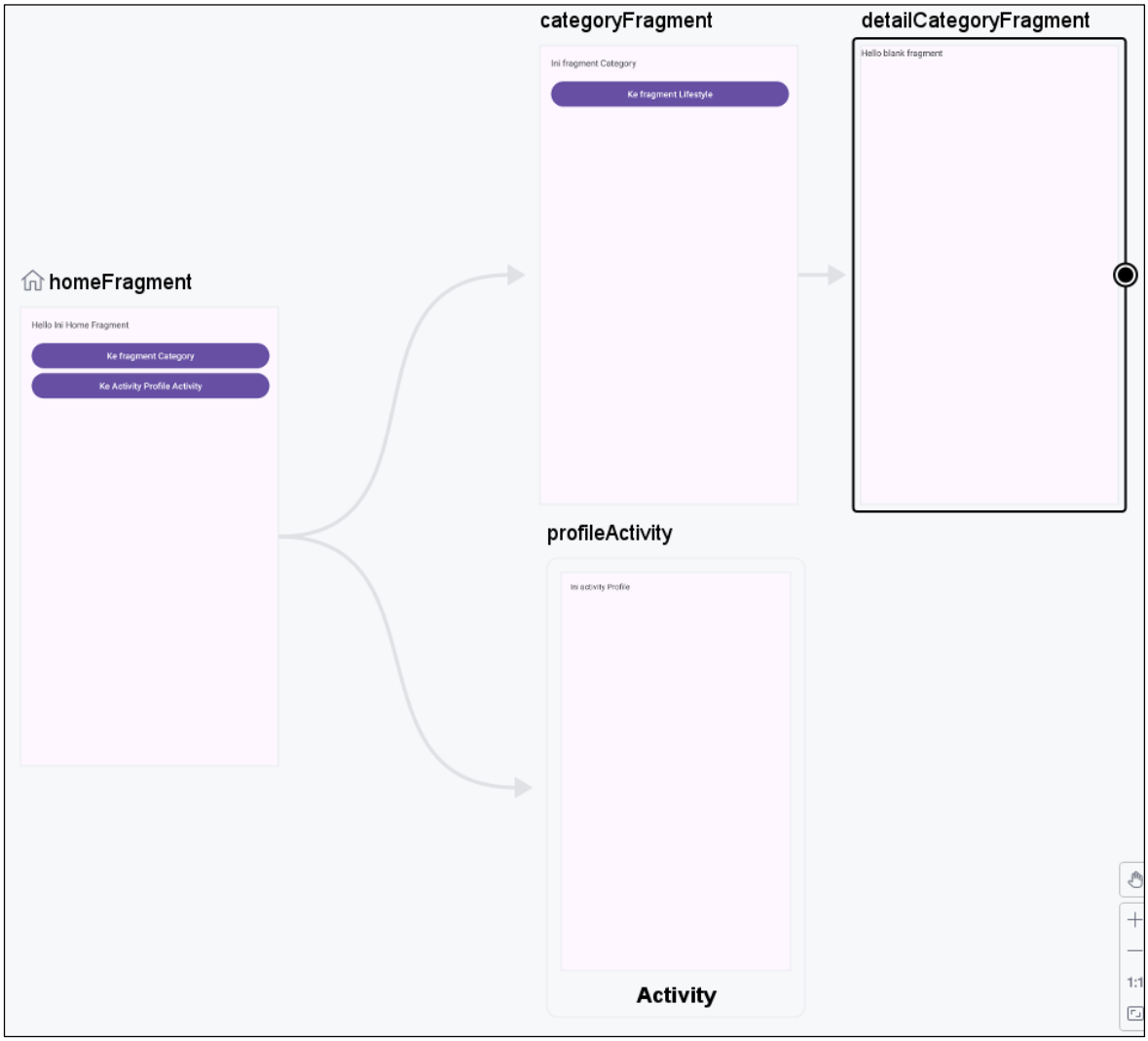
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp"
        android:text="@string/this_category" />

    <Button
        android:id="@+id/btn_category_lifestyle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:text="@string/category_lifestyle" />

</LinearLayout>
```

13. Buat fragment baru dengan cara addDestination, kemudian pilih Fragment(Blank) lalu berilah nama DetailCategoryFragment dan klik Finish. Kemudian buat action dari categoryFragment ke detailCategoryFragment.



Kemudian buat terlebih dahulu layout di fragment_detail_category menjadi seperti ini:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".DetailCategoryFragment">
```

```
    <TextView
        android:id="@+id/tv_category_name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp"
        android:text="@string/category_name" />

    <TextView
        android:id="@+id/tv_category_description"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp"
        android:text="@string/category_description" />

</LinearLayout>
```

14. Setelah itu mulai proses pengiriman data dengan bundle dengan masuk ke kelas CategoryFragment dan tambahkan kode berikut:

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.navigation.findNavController
import
id.ac.polbeng.depandienda.navigationexample.databinding.FragmentCategoryBi
nding

class CategoryFragment : Fragment() {
    private var _binding: FragmentCategoryBinding? = null
    private val binding get() = _binding!!

    companion object {
        val EXTRA_NAME = "extra_name"
        val EXTRA_STOCK = "extra_stock"
    }

    override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
                              savedInstanceState: Bundle?): View {
        // Inflate the layout for this fragment
        _binding = FragmentCategoryBinding.inflate(inflater, container,
false)
        val view = binding.root
```

```
        return view
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        binding.btnCategoryLifestyle.setOnClickListener { view ->
            val mBundle = Bundle()
            mBundle.putString(EXTRA_NAME, "Lifestyle")
            mBundle.putLong(EXTRA_STOCK, 7)

view.findNavController().navigate(R.id.action_categoryFragment_to_detailCa
tegoryFragment, mBundle)
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        _binding = null
    }
}
```

15. Kemudian untuk menerima datanya, masuk ke kelas DetailCategoryFragment dan tambahkan kode berikut:

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import
id.ac.polbeng.depandienda.navigationexample.databinding.FragmentDetailCate
goryBinding

class DetailCategoryFragment : Fragment() {

    private var _binding: FragmentDetailCategoryBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
                              savedInstanceState: Bundle?): View {
        // Inflate the layout for this fragment
        _binding = FragmentDetailCategoryBinding.inflate(inflater,
container, false)
        val view = binding.root
```

```
        return view
    }

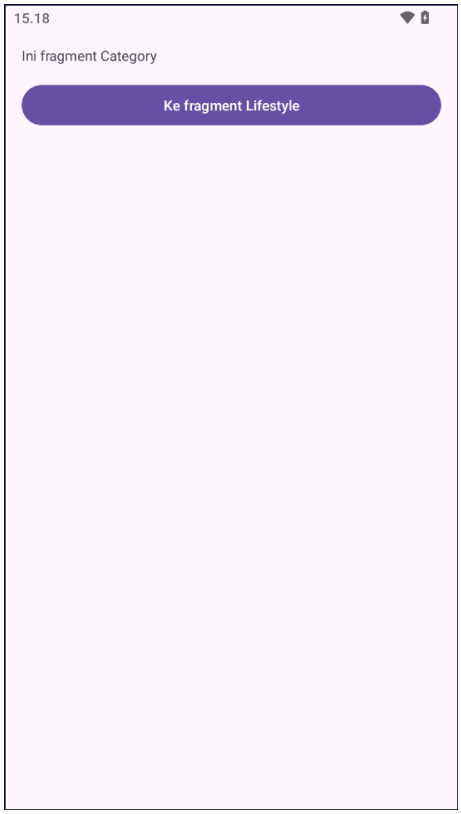
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

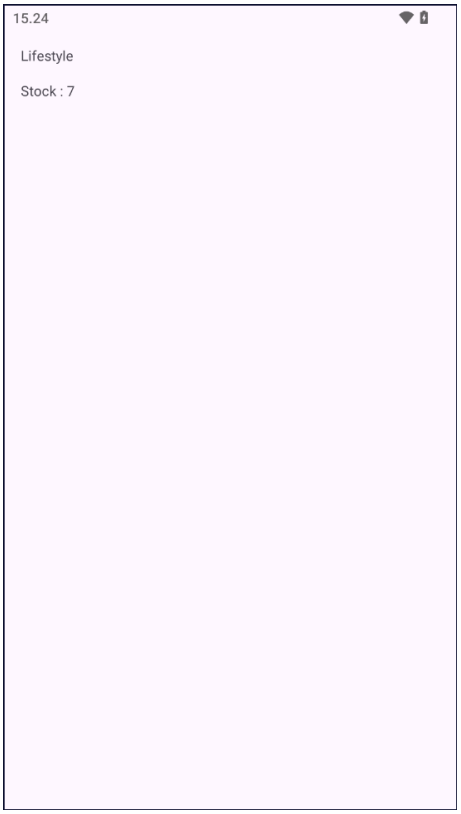
        val dataName = arguments?.getString(CategoryFragment.EXTRA_NAME)
        val dataDescription =
arguments?.getLong(CategoryFragment.EXTRA_STOCK)

        binding.tvCategoryName.text = dataName
        binding.tvCategoryDescription.text = "Stock : $dataDescription"
    }

    override fun onDestroy() {
        super.onDestroy()
        _binding = null
    }
}
```

Kemudian coba jalankan kembali aplikasi, maka ia akan terlihat seperti ini:



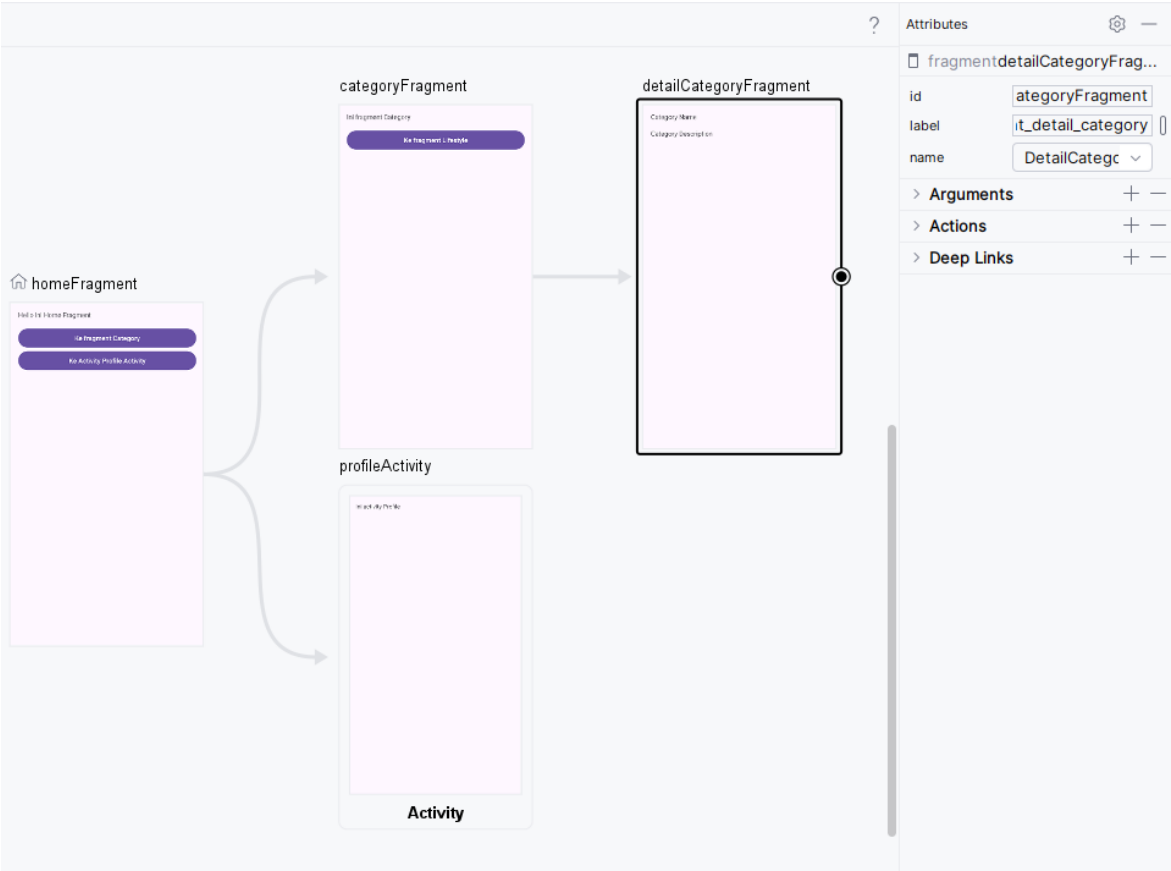


Mengirim Data dengan SafeArg

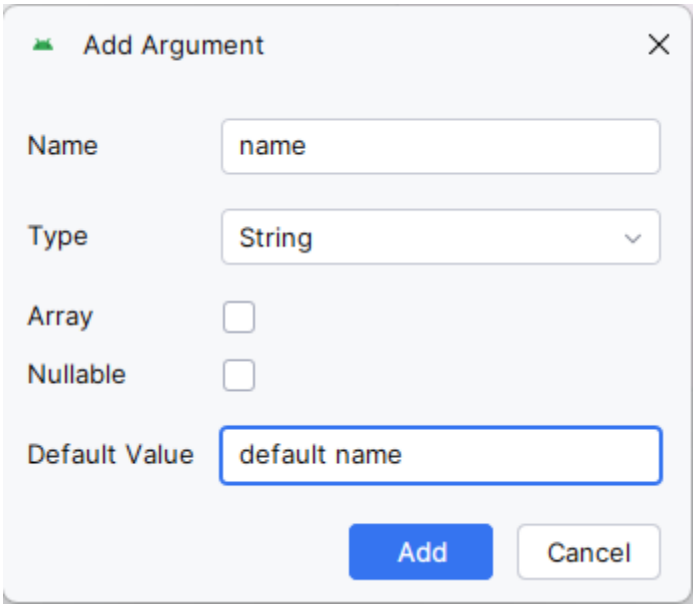
Pengiriman data dengan menggunnakan bundle bisa jadi tidak aman. Hal ini bisa terjadi apabila saat mengirim dan menerima data berbeda tipe datanya, misalnya ketika mengirim menggunakan putString namun saat menerima menggunakan getInt, sehingga akan menghasilkan nilai null yang

membuat error **null pointer exception (NPE)**. Sebagai solusinya, Anda bisa menggunakan **SafeArg** yang merupakan salah satu fitur di Navigation untuk melakukan pengiriman data yang aman alias **null-safety**.

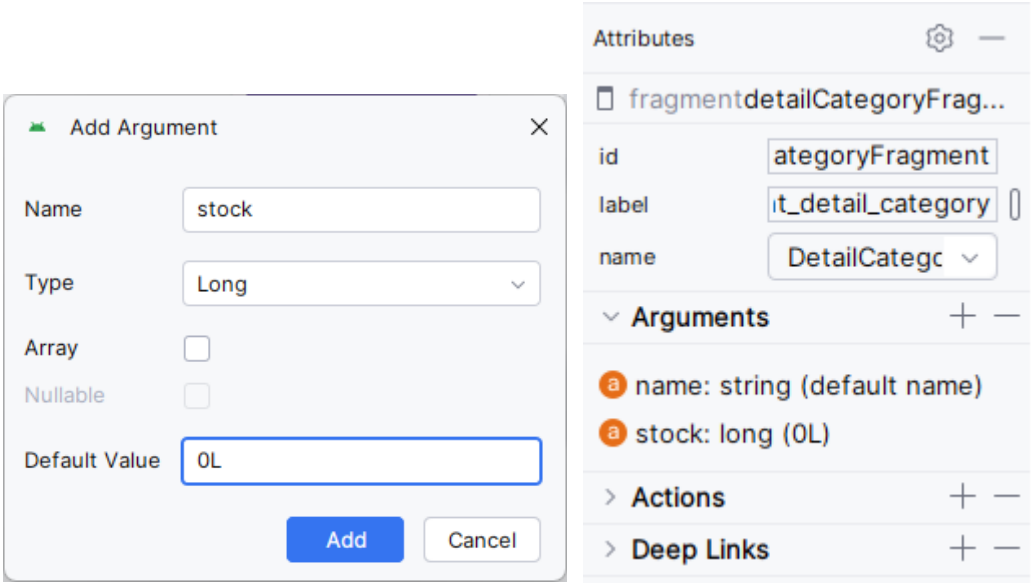
16. Untuk menambahkan data argument baru dengan SafeArg, klik icon add arguments bertanda plus.



kemudian isi nama ,jenis data yang dikirim, default value, dengan name , type String, dan default name.



Buat sekali lagi dengan nama stock, jenis data Long, dan default value 0L sehingga argument menjadi seperti ini:



maka bagian tab Code/Text pada main_navigation akan menjadi seperti ini:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main_navigation"
    app:startDestination="@id/homeFragment">
    ...
    <fragment
        android:id="@+id/detailCategoryFragment"
```

```
android:name="id.ac.polbeng.depandienda.navigationexample.DetailCategoryFr
agment"
        android:label="fragment_detail_category"
        tools:layout="@layout/fragment_detail_category" >
        <argument
            android:name="name"
            app:argType="string"
            android:defaultValue="default name" />
        <argument
            android:name="stock"
            app:argType="long"
            android:defaultValue="0L" />
    </fragment>
</navigation>
```

17. Kemudian Anda perlu menambahkan library SafeArgs di bagian: build.gradle.kts(Project)

```
// Top-level build file where you can add configuration options common to
all sub-projects/modules.
buildscript {
    dependencies {
        classpath ("androidx.navigation:navigation-safe-args-gradle-
plugin:2.9.7")
    }
}
plugins {
    alias(libs.plugins.android.application) apply false
}
```

Tambahkan juga pemanggilan librarynya di build.gradle.kts(Module: app).

```
plugins {
    alias(libs.plugins.android.application)
    id ("androidx.navigation.safeargs")
}

android {
...
}
```

Jangan lupa untuk sync project / rebuild project, maka akan otomatis tercipta file baru, yaitu CategoryFragmentDirections dan DetailCategoryFragmentArgs.

18. Kemudian ganti metode yang sebelumnya menggunakan bundle. Ubah dengan menerapkan SafeArg. Yang pertama yaitu ubah kode pada CategoryFragment sehingga menjadi seperti berikut:

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
```

```
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.navigation.findNavController
import
id.ac.polbeng.depandienda.navigationexample.databinding.FragmentCategoryBi
nding

class CategoryFragment : Fragment() {
    private var _binding: FragmentCategoryBinding? = null
    private val binding get() = _binding!!

    companion object {
        val EXTRA_NAME = "extra_name"
        val EXTRA_STOCK = "extra_stock"
    }

    override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
                              savedInstanceState: Bundle?): View {
        // Inflate the layout for this fragment
        _binding = FragmentCategoryBinding.inflate(inflater, container,
false)
        val view = binding.root
        return view
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        binding.btnCategoryLifestyle.setOnClickListener { view ->
            val toDetailCategoryFragment =
CategoryFragmentDirections.actionCategoryFragmentToDetailCategoryFragment(
)
            toDetailCategoryFragment.name = "Lifestyle"
            toDetailCategoryFragment.stock = 7
            view.findNavController().navigate(toDetailCategoryFragment)
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        _binding = null
    }
}
```

Kemudian pada DetailCategoryFragment ganti kodenya menjadi seperti berikut:

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import
id.ac.polbeng.depandienda.navigationexample.databinding.FragmentDetailCate
goryBinding

class DetailCategoryFragment : Fragment() {

    private var _binding: FragmentDetailCategoryBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
                              savedInstanceState: Bundle?): View {
        // Inflate the layout for this fragment
        _binding = FragmentDetailCategoryBinding.inflate(inflater,
container, false)
        val view = binding.root
        return view
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val dataName = DetailCategoryFragmentArgs.fromBundle(arguments as
Bundle).name
        val dataDescription =
DetailCategoryFragmentArgs.fromBundle(arguments as Bundle).stock

        binding.tvCategoryName.text = dataName
        binding.tvCategoryDescription.text = "Stock : $dataDescription"
    }

    override fun onDestroy() {
        super.onDestroy()
        _binding = null
    }
}
```

Kemudian jalankan kembali aplikasinya. Memang tidak ada perubahan yang berarti dengan cara menggunakan cara bundle, namun cara ini membuat proses pengiriman data menjadi lebih aman.

Mengimplementasikan Popup Behavior

Pop Up Behaviour adalah tingkah laku aplikasi ketika ditekan tombol back. Jadi Anda bisa mengatur apakah seperti biasa, langsung keluar atau bisa juga diatur menuju ke fragment mana.

19. Pertama, tambahkan tombol di dalam fragment_detail_category seperti berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    ...
    tools:context=".DetailCategoryFragment">

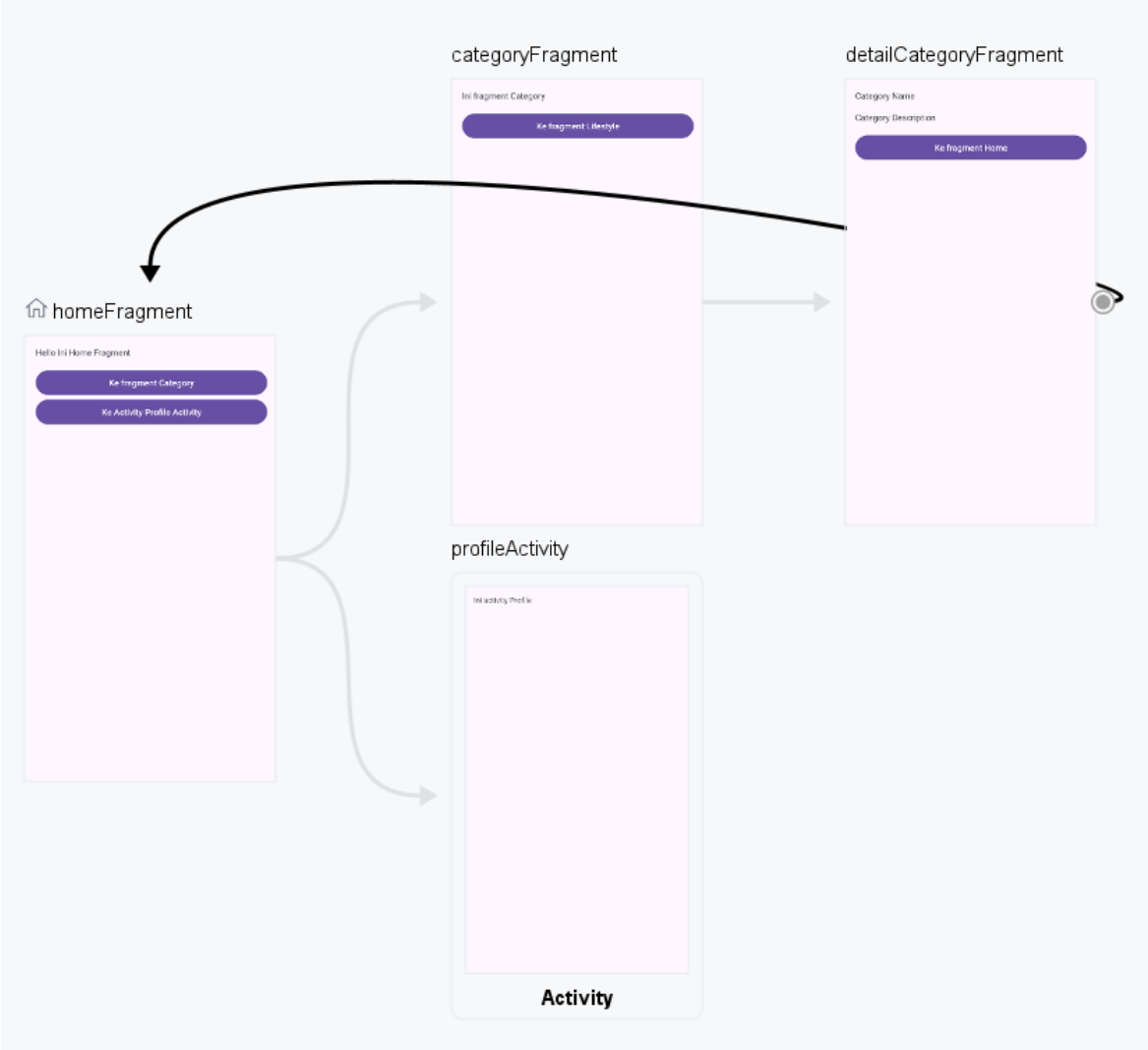
    <TextView
        android:id="@+id/tv_category_name" ... />

    <TextView
        android:id="@+id/tv_category_description" ... />

    <Button
        android:id="@+id/btn_home"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/to_home" />

</LinearLayout>
```

Kemudian buat action dari detailCategoryFragment ke homeFragment dengan cara klik lingkaran bulat pada detailCategoryFragment kemudian tarik garis ke homeFragment.



Dan pada tab Code/Text akan menjadi seperti ini:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    ...
    app:startDestination="@id/homeFragment">
    <fragment
        ...
    </fragment>
    <fragment
        ...
    </fragment>
    <activity
        android:id="@+id/profileActivity" ... />
    <fragment
        android:id="@+id/detailCategoryFragment" ... >
        <argument
            android:name="name".../>
```

```
        <argument
            android:name="stock" ... />
        <action
           android:id="@+id/action_detailCategoryFragment_to_homeFragment"
            app:destination="@id/homeFragment" />
    </fragment>
</navigation>
```

Kemudian tambahkan di DetailCategoryFragment kode berikut untuk memanggil aksi yang sudah dibuat:

```
package id.ac.polbeng.depandienda.navigationexample

import android.os.Bundle
...

class DetailCategoryFragment : Fragment() {

    private var _binding: FragmentDetailCategoryBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
                              savedInstanceState: Bundle?): View {
        ...
    }

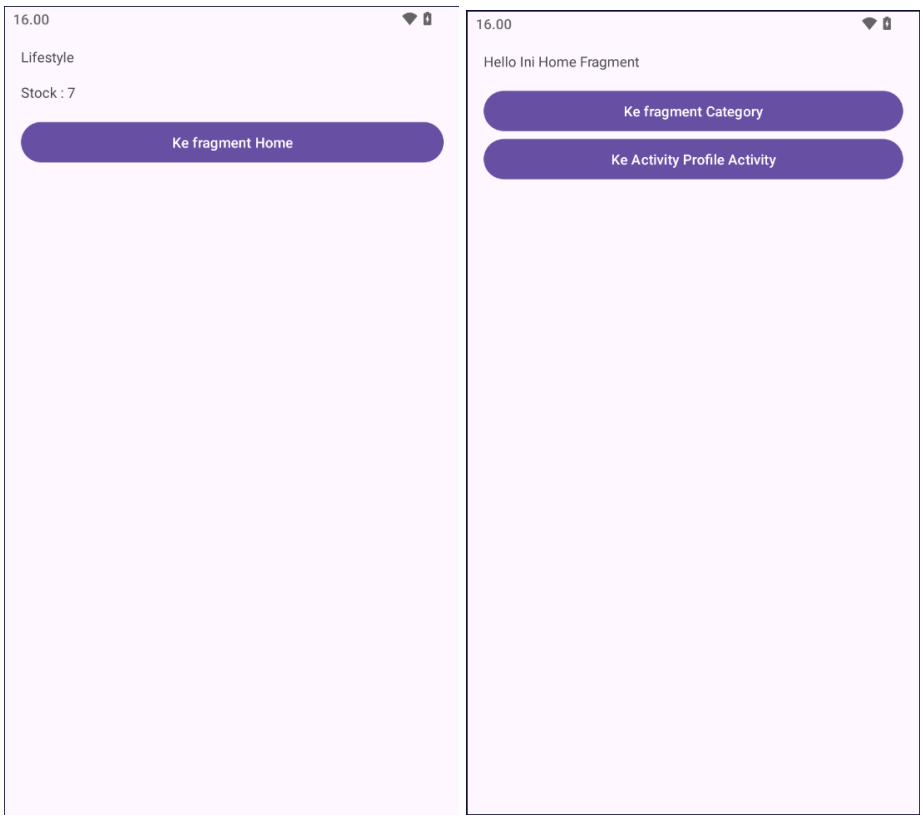
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        ...

        binding.tvCategoryName.text = dataName
        binding.tvCategoryDescription.text = "Stock : $dataDescription"

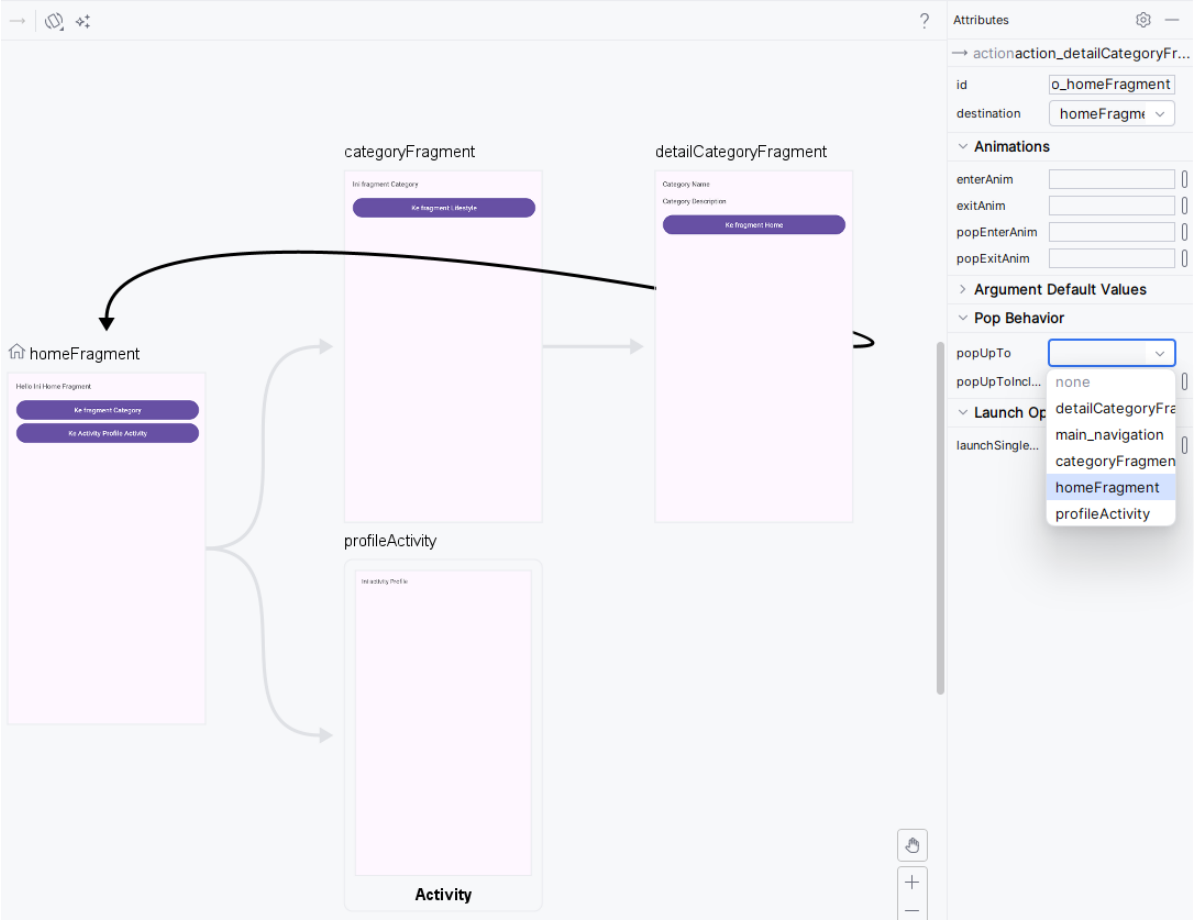
        binding.btnHome.setOnClickListener(
Navigation.createNavigateOnClickListener(R.id.action_detailCategoryFragmen
t_to_homeFragment)
        )
    }

    ...
}
```

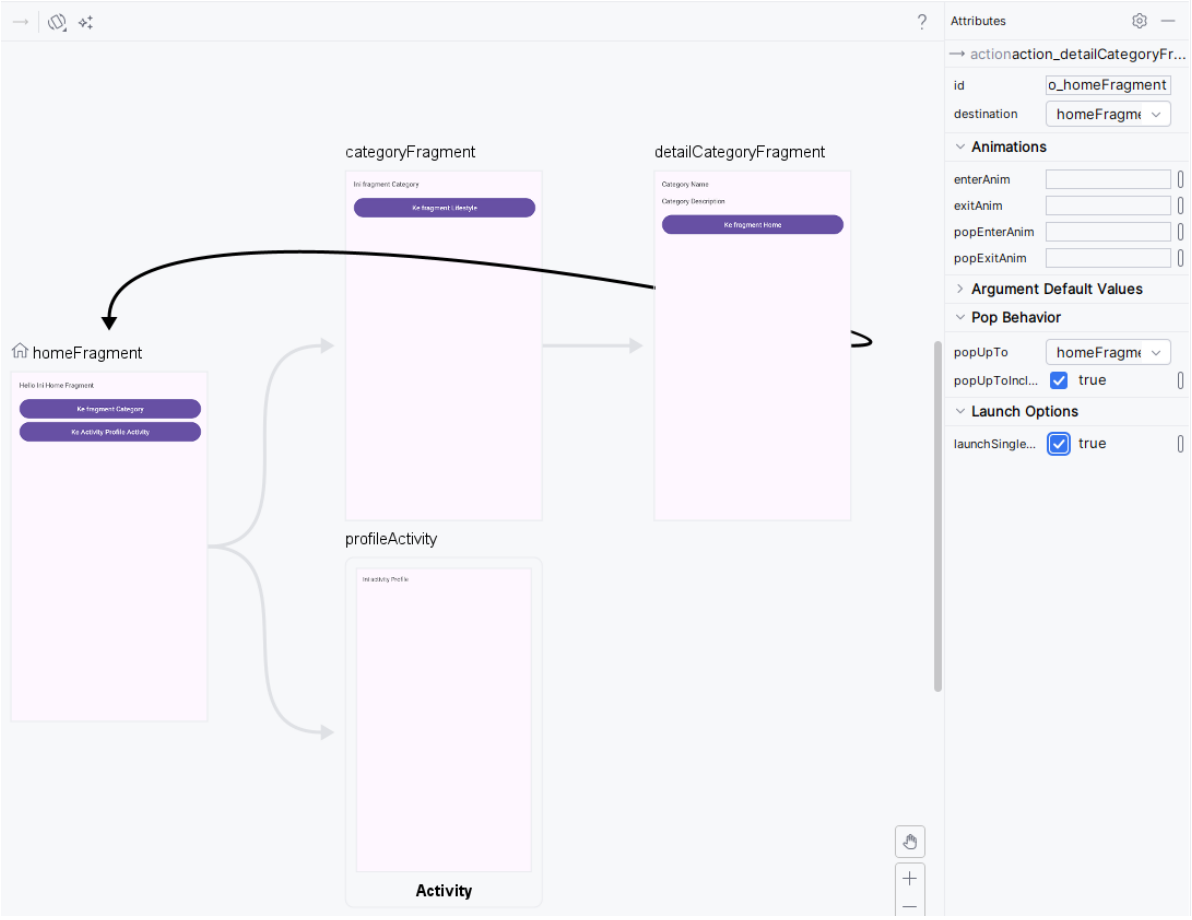
Sampai di sini, cobalah untuk menjalankan aplikasi Anda terlebih dahulu.



Silakan coba masuk sampai kembali ke halaman HomeFragment, lalu tekan tombol Back Button. Yang terjadi saat ini adalah Anda harus kembali ke halaman-halaman sebelumnya untuk keluar dari aplikasi. Padahal ketika tombol back di halaman Home ditekan, biasanya pengguna akan langsung keluar dari aplikasi. Untuk mengatasi hal tersebut, gunakanlah Pop Up Behaviour.



Untuk mengatasinya, tekanlah action_detailCategoryFragment_to_homeFragment kemudian centang pada launchSingleTop, isi popUpTo dengan homeFragment, dan centang pada popUpToInclusive. Ini berfungsi seperti Intent.FLAG_ACTIVITY_CLEAR_TOP.



Dan kode pada tab Code/Text akan tertambah seperti ini:

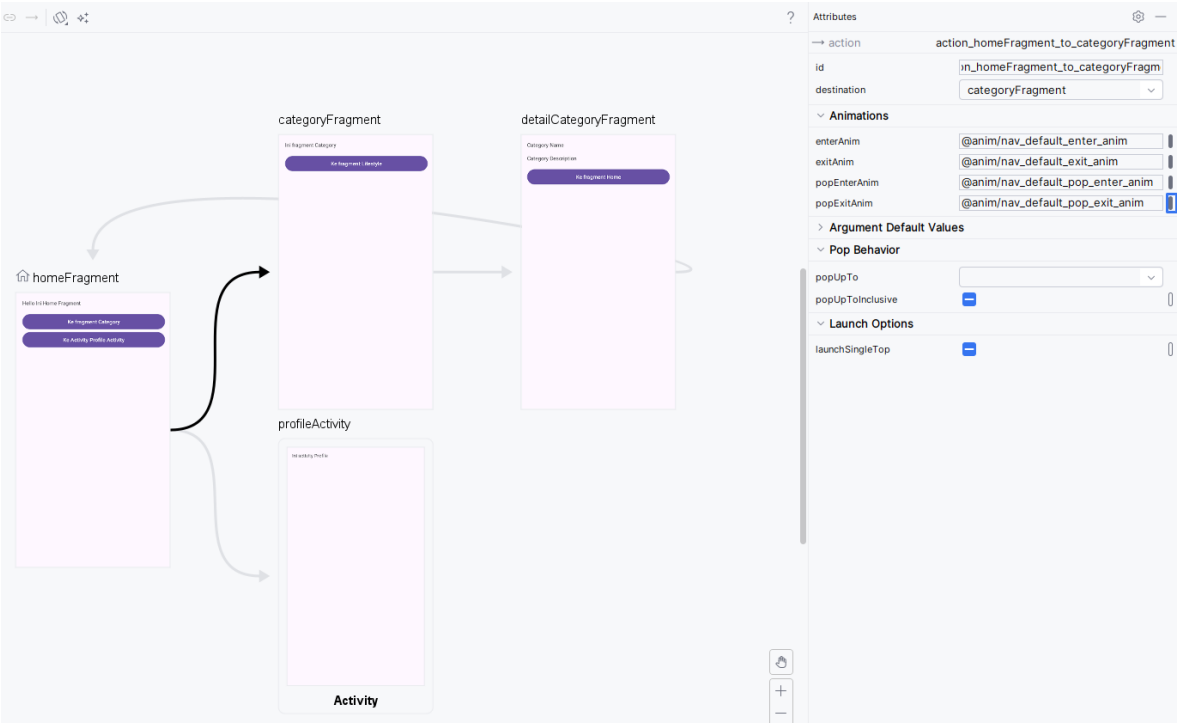
```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    ...
    app:startDestination="@id/homeFragment">
    ...
    <fragment
        android:id="@+id/detailCategoryFragment"... >
        ...
        <action
           android:id="@+id/action_detailCategoryFragment_to_homeFragment"
            app:destination="@id/homeFragment"
            app:launchSingleTop="true"
            app:popUpTo="@id/homeFragment"
            app:popUpToInclusive="true" />
    </fragment>
</navigation>
```

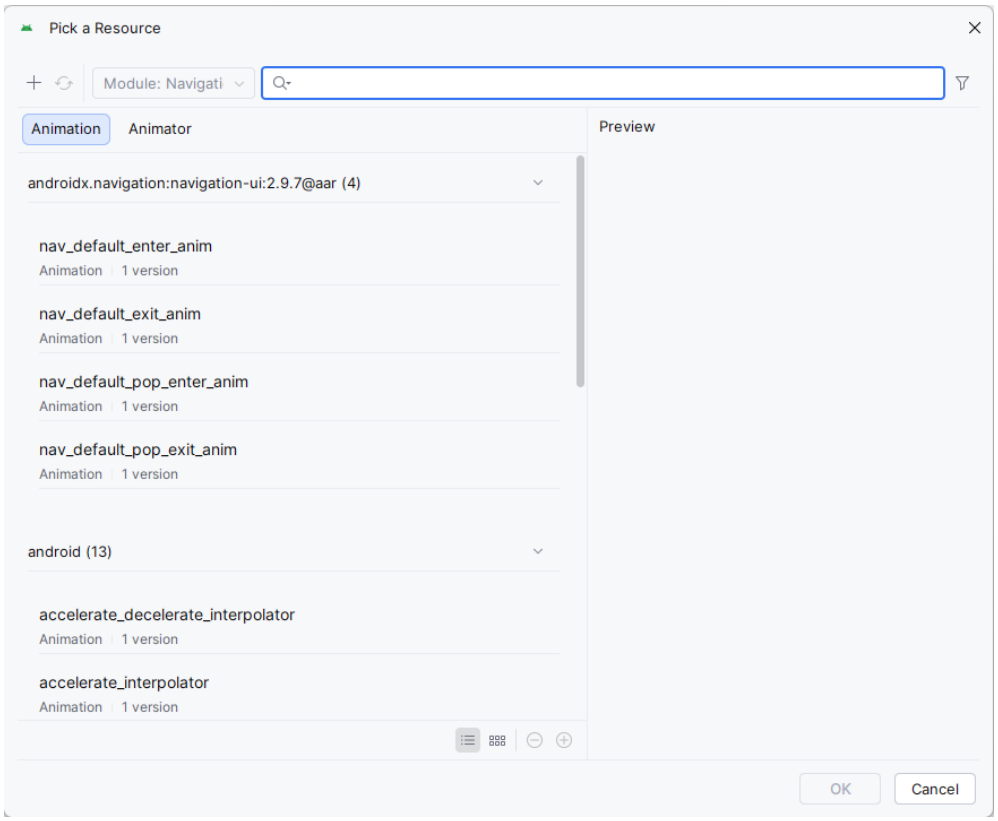
Jalankanlah aplikasi maka perhatikan perbedaannya setelah di klik button ke fragment home maka ketika diklik tombol back di fragment Home maka aplikasi akan keluar secara langsung. Hal ini dikarena fragment yang sebelumnya telah dibersihkan. Ini juga bisa digunakan saat Anda membuat Splash Screen.

Mengimplementasikan Animasi

Anda bisa menambahkan animasi saat transisi antar Activity dengan mudah menggunakan Navigation. Berikut langkah-langkahnya:

20. Untuk menambahkan animasi, cukup pilih action yang akan ditambahkan animasi, kemudian tinggal pilih animasinya.



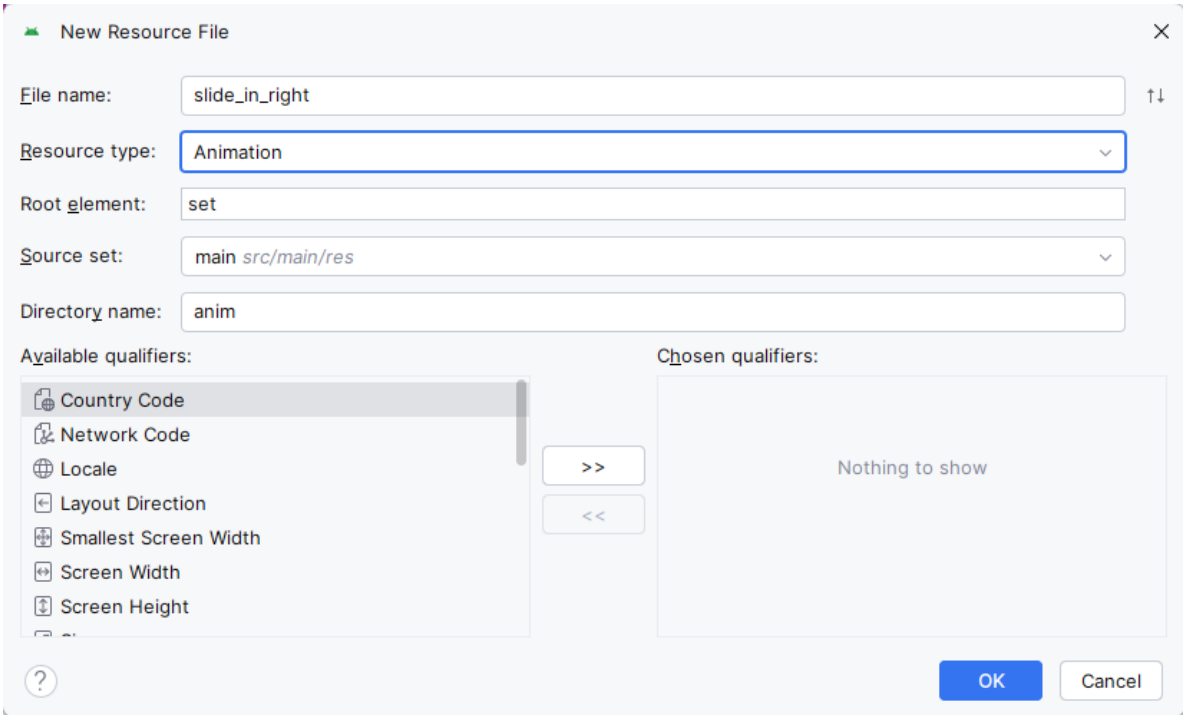


Maka pada tab Code/Text akan tertambah kode seperti ini:

```
<fragment
    android:id="@+id/homeFragment"

android:name="id.ac.polbeng.depandienda.navigationexample.HomeFragment"
    android:label="fragment_home"
    tools:layout="@layout/fragment_home" >
    <action
        android:id="@+id/action_homeFragment_to_categoryFragment"
        app:destination="@id/categoryFragment"
        app:enterAnim="@anim/nav_default_enter_anim"
        app:exitAnim="@anim/nav_default_exit_anim"
        app:popEnterAnim="@anim/nav_default_pop_enter_anim"
        app:popExitAnim="@anim/nav_default_pop_exit_anim" />
    <action
        android:id="@+id/action_homeFragment_to_profileActivity"
        app:destination="@id/profileActivity" />
</fragment>
```

21. Kemudian Anda juga bisa menambahkan animasi sendiri secara custom, caranya yaitu Klik kanan pada res → New → Android Resource File. Kemudian berilah nama slide_in_right dan ganti Resource Type menjadi Animation.



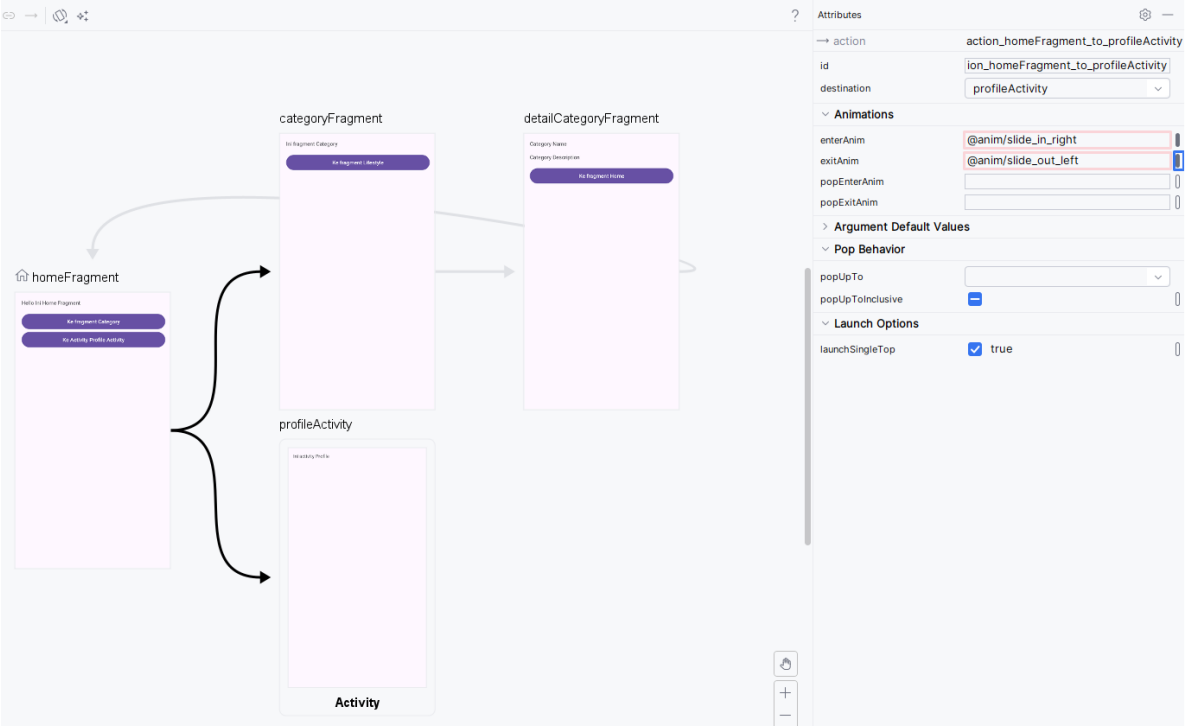
Selanjutnya akan tercipta berkas baru di res → anim. Kemudian ubah kode pada slide_in_right menjadi seperti berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate android:fromXDelta="100%" android:toXDelta="0%"
        android:fromYDelta="0%" android:toYDelta="0%"
        android:duration="700"/>
</set>
```

22. Kemudian buat sekali lagi dengan cara Klik kanan pada res → New → Android Resource File. Kemudian beri nama slide_out_left dan ganti Resource Type menjadi Animation. Kemudian ubah kode di dalamnya menjadi:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate android:fromXDelta="0%" android:toXDelta="-100%"
        android:fromYDelta="0%" android:toYDelta="0%"
        android:duration="700"/>
</set>
```

23. Jika sudah, sekarang pilih animasi yang sudah dibuat pada action yang dari HomeFragment ke ProfileActivity dan centang pada launchSingleTop.



Maka pada tab Code/Text akan tertambah kode berikut:

```
<fragment
    android:id="@+id/homeFragment"
   android:name="id.ac.polbeng.depandienda.navigationexample.HomeFragment"
    android:label="fragment_home"
    tools:layout="@layout/fragment_home" >
    <action
        android:id="@+id/action_homeFragment_to_categoryFragment"
        app:destination="@id/categoryFragment"
        app:enterAnim="@anim/nav_default_enter_anim"
        app:exitAnim="@anim/nav_default_exit_anim"
        app:popEnterAnim="@anim/nav_default_pop_enter_anim"
        app:popExitAnim="@anim/nav_default_pop_exit_anim" />
    <action
        android:id="@+id/action_homeFragment_to_profileActivity"
        app:destination="@id/profileActivity"
        app:enterAnim="@anim/slide_in_right"
        app:exitAnim="@anim/slide_out_left"
        app:launchSingleTop="true" />
</fragment>
```

Maka kode lengkap pada main_navigation sekarang menjadi seperti ini:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main_navigation"
```

```
    app:startDestination="@id/homeFragment">
    <fragment
        android:id="@+id/homeFragment"

android:name="id.ac.polbeng.depandienda.navigationexample.HomeFragment"
        android:label="fragment_home"
        tools:layout="@layout/fragment_home" >
        <action
            android:id="@+id/action_homeFragment_to_categoryFragment"
            app:destination="@id/categoryFragment"
            app:enterAnim="@anim/nav_default_enter_anim"
            app:exitAnim="@anim/nav_default_exit_anim"
            app:popEnterAnim="@anim/nav_default_pop_enter_anim"
            app:popExitAnim="@anim/nav_default_pop_exit_anim" />
        <action
            android:id="@+id/action_homeFragment_to_profileActivity"
            app:destination="@id/profileActivity"
            app:enterAnim="@anim/slide_in_right"
            app:exitAnim="@anim/slide_out_left"
            app:launchSingleTop="true" />
    </fragment>
    <fragment
        android:id="@+id/categoryFragment"

android:name="id.ac.polbeng.depandienda.navigationexample.CategoryFragment
"
        android:label="fragment_category"
        tools:layout="@layout/fragment_category" >
        <action

android:id="@+id/action_categoryFragment_to_detailCategoryFragment"
            app:destination="@id/detailCategoryFragment" />
    </fragment>
    <activity
        android:id="@+id/profileActivity"

android:name="id.ac.polbeng.depandienda.navigationexample.ProfileActivity"
        android:label="activity_profile"
        tools:layout="@layout/activity_profile" />
    <fragment
        android:id="@+id/detailCategoryFragment"

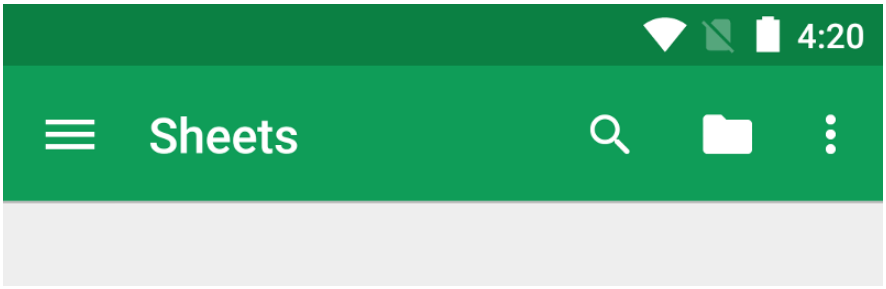
android:name="id.ac.polbeng.depandienda.navigationexample.DetailCategoryFr
agment"
        android:label="fragment_detail_category"
        tools:layout="@layout/fragment_detail_category" >
        <argument
            android:name="name"
            app:argType="string"
```

```
            android:defaultValue="default name" />
        <argument
            android:name="stock"
            app:argType="long"
            android:defaultValue="0L" />
        <action

android:id="@+id/action_detailCategoryFragment_to_homeFragment"
            app:destination="@id/homeFragment"
            app:launchSingleTop="true"
            app:popUpTo="@id/homeFragment"
            app:popUpToInclusive="true" />
    </fragment>
</navigation>
```

Jalankanlah kembali aplikasi yang Anda buat, maka akan terdapat animasi untuk ke halam Fragment Category dan Profile Activity.

AppBar



AppBar adalah salah satu komponen penting di dalam struktur sebuah aplikasi Android. Biasanya berada di bagian atas aplikasi untuk menampilkan judul halaman. AppBar merupakan rancangan yang dianjurkan dalam membuat aplikasi. Bila kita menggunakannya dengan baik, maka kita dapat mempertahankan konsistensi aplikasi. Tentunya ini akan berimbas positif pada user experience.

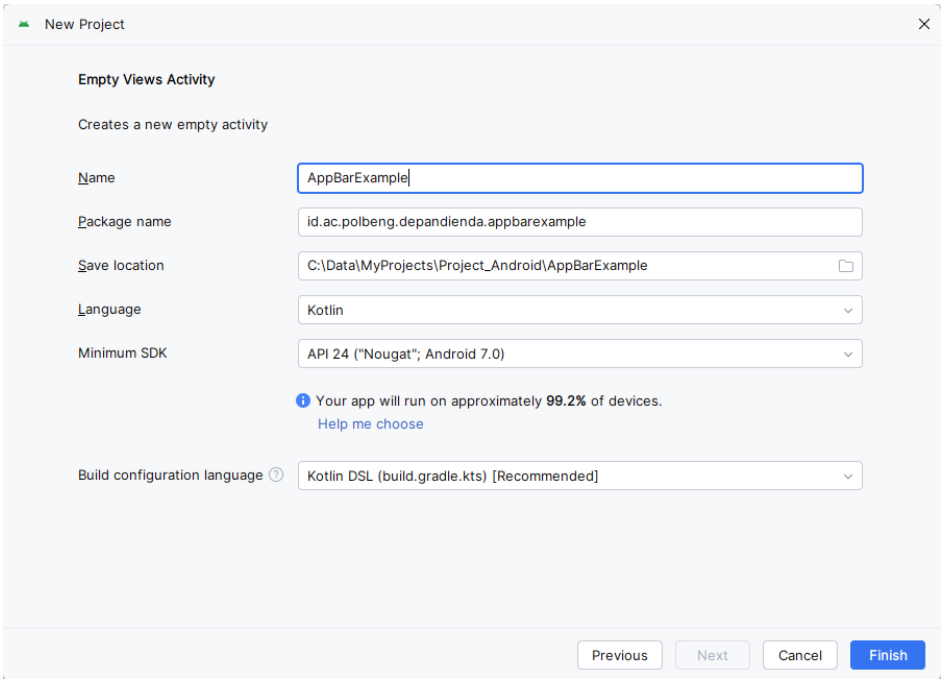
Beberapa fungsi penting dari AppBar adalah:

1. Memberikan ruangan kepada identitas app dan juga mengindikasikan lokasi pengguna pada saat menjalankan aplikasi.
2. Akses ke beberapa action penting dengan lebih mudah seperti action pencarian.
3. Sebagai navigation dan switch antar view (tampilan).

Di dalam Android, terdapat dua jenis App Bar, yakni ActionBar dan Toolbar.

Latihan Mengimplementasikan AppBar

1. Buatlah project baru dengan nama AppBarExample.



2. Jangan lupa untuk mengaktifkan view-binding dengan menambahkan kode berikut pada build.gradle di level module build.gradle(Module: app), dan tambahkan juga dependency atau library ***coordinatorlayout***

```
android {
    ...
    buildFeatures {
        viewBinding = true
    }
}
dependencies {
    ...
    implementation(libs.androidx.coordinatorlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
}
```

3. Selanjutnya editlah file strings.xml pada folder values, tambahkan beberapa variabel dan nilai string sebagai berikut:

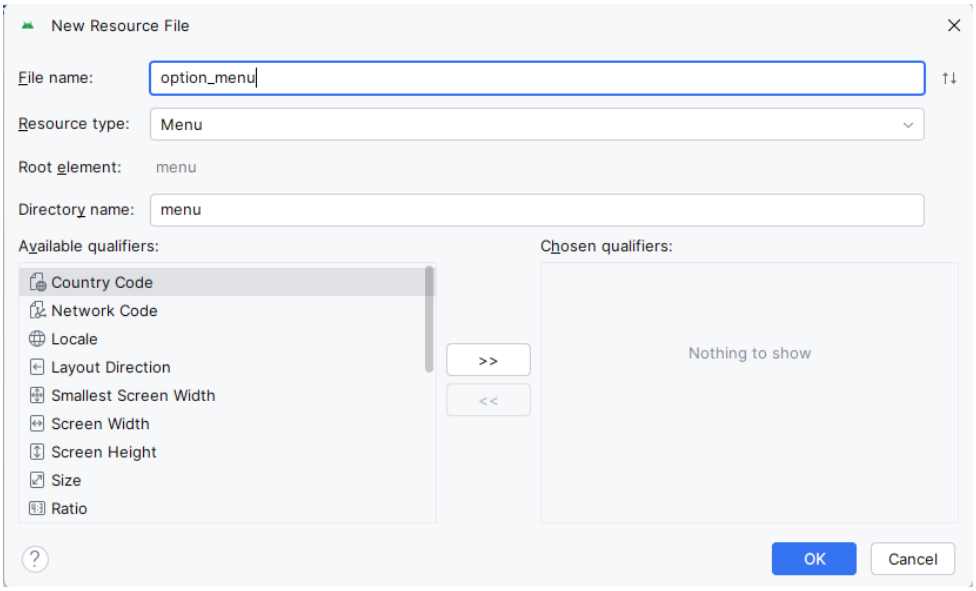
```
<resources>
    <string name="app_name">AppBarExample</string>
    <string name="hello_blank_fragment">Hello blank fragment</string>
    <string name="this_activity">Ini adalah menu activity</string>
    <string name="menu_1">Menu 1</string>
    <string name="menu_2">Menu 2</string>
    <string name="large_text">
        "Material is the metaphor.\n\n"
        "A material metaphor is the unifying theory of a rationalized
```

```
space and a system of motion."
        "The material is grounded in tactile reality, inspired by the
study of paper and ink, yet "
        "technologically advanced and open to imagination and magic.\n"
        "Surfaces and edges of the material provide visual cues that are
grounded in reality. The "
        "use of familiar tactile attributes helps users quickly
understand affordances. Yet the "
        "flexibility of the material creates new affordances that
supercede those in the physical "
        "world, without breaking the rules of physics.\n"
        "The fundamentals of light, surface, and movement are key to
conveying how objects move, "
        "interact, and exist in space and in relation to each other.
Realistic lighting shows "
        "seams, divides space, and indicates moving parts.\n\n"
        "Bold, graphic, intentional.\n\n"
        "The foundational elements of print based design typography,
grids, space, scale, color, "
        "and use of imagery guide visual treatments. These elements do
far more than please the "
        "eye. They create hierarchy, meaning, and focus. Deliberate color
choices, edge to edge "
        "imagery, large scale typography, and intentional white space
create a bold and graphic "
        "interface that immerse the user in the experience.\n"
        "An emphasis on user actions makes core functionality immediately
apparent and provides "
        "waypoints for the user.\n\n"
        "Motion provides meaning.\n\n"
        "Motion respects and reinforces the user as the prime mover.
Primary user actions are "
        "inflection points that initiate motion, transforming the whole
design.\n"
        "All action takes place in a single environment. Objects are
presented to the user without "
        "breaking the continuity of experience even as they transform and
reorganize.\n"
        "Motion is meaningful and appropriate, serving to focus attention
and maintain continuity. "
        "Feedback is subtle yet clear. Transitions are efficient yet
coherent.\n\n"
        "3D world.\n\n"
        "The material environment is a 3D space, which means all objects
have x, y, and z "
        "dimensions. The z-axis is perpendicularly aligned to the plane
of the display, with the "
        "positive z-axis extending towards the viewer. Every sheet of
material occupies a single "
```

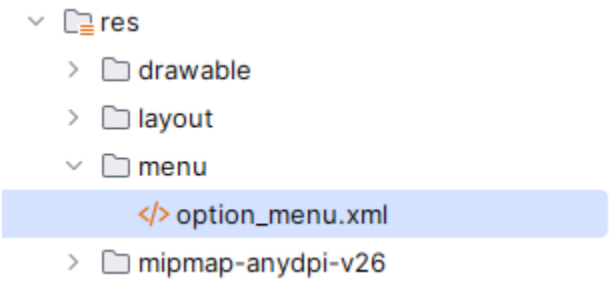
```
        "position along the z-axis and has a standard 1dp thickness.\n"
        "On the web, the z-axis is used for layering and not for
perspective. The 3D world is "
        "emulated by manipulating the y-axis.\n\n"
        "Light and shadow.\n\n"
        "Within the material environment, virtual lights illuminate the
scene. Key lights create "
        "directional shadows, while ambient light creates soft shadows
from all angles.\n"
        "Shadows in the material environment are cast by these two light
sources. In Android "
        "development, shadows occur when light sources are blocked by
sheets of material at "
        "various positions along the z-axis. On the web, shadows are
depicted by manipulating the "
        "y-axis only. The following example shows the card with a height
of 6dp.\n\n"
        "Resting elevation.\n\n"
        "All material objects, regardless of size, have a resting
elevation, or default elevation "
        "that does not change. If an object changes elevation, it should
return to its resting "
        "elevation as soon as possible.\n\n"
        "Component elevations.\n\n"
        "The resting elevation for a component type is consistent across
apps (e.g., FAB elevation "
        "does not vary from 6dp in one app to 16dp in another app).\n"
        "Components may have different resting elevations across
platforms, depending on the depth "
        "of the environment (e.g., TV has a greater depth than mobile or
desktop).\n\n"
        "Responsive elevation and dynamic elevation offsets.\n\n"
        "Some component types have responsive elevation, meaning they
change elevation in response "
        "to user input (e.g., normal, focused, and pressed) or system
events. These elevation "
        "changes are consistently implemented using dynamic elevation
offsets.\n"
        "Dynamic elevation offsets are the goal elevation that a
component moves towards, relative "
        "to the component’s resting state. They ensure that elevation
changes are consistent "
        "across actions and component types. For example, all components
that lift on press have "
        "the same elevation change relative to their resting
elevation.\n"
        "Once the input event is completed or cancelled, the component
will return to its resting "
        "elevation.\n\n"
```

```
        "Avoiding elevation interference.\n\n"
        "Components with responsive elevations may encounter other
components as they move between "
        "their resting elevations and dynamic elevation offsets. Because
material cannot pass "
        "through other material, components avoid interfering with one
another any number of ways, "
        "whether on a per component basis or using the entire app
layout.\n"
        "On a component level, components can move or be removed before
they cause interference. "
        "For example, a floating action button (FAB) can disappear or
move off screen before a "
        "user picks up a card, or it can move if a snackbar appears.\n"
        "On the layout level, design your app layout to minimize
opportunities for interference. "
        "For example, position the FAB to one side of stream of a cards
so the FAB won't interfere "
        "when a user tries to pick up one of cards.\n\n"
    </string>
</resources>
```

4. Setelah itu buatlah berkas xml baru pada directory menu dengan nama berkas option_menu.xml. Cara membuat menu baru yaitu klik kanan pada res → new → Android resource file. Ganti nama berkas dan directory seperti gambar berikut:



Sehingga akan muncul file baru pada folder menu seperti ini:



5. Tambahkan dua menu pada option_menu.xml menjadi seperti berikut :

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <group android:menuCategory="container">
        <item
            android:id="@+id/menu1"
            android:icon="@drawable/baseline_announcement_24"
            android:title="@string/menu_1"
            app:showAsAction="ifRoom" />
        <item
            android:id="@+id/menu2"
            android:title="@string/menu_2"
            app:showAsAction="never" />
    </group>
</menu>
```

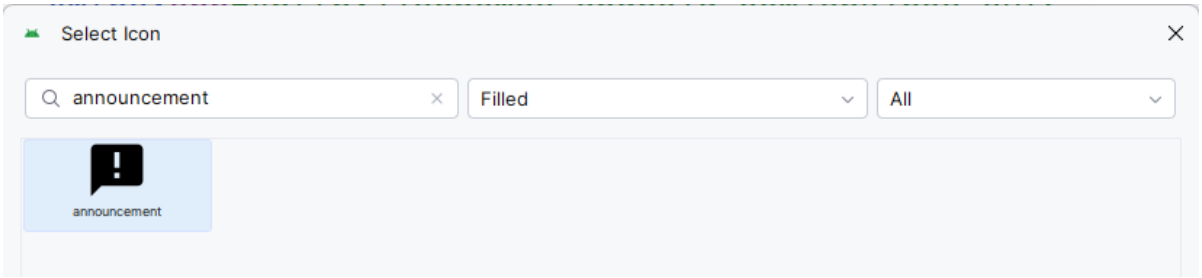
Tag <group> dapat digunakan untuk mengelompokkan item yang akan ditampilkan pada AppBar. Perhatikan tag <item>. Dengan menggunakan android:icon kita dapat mengganti ikon pada item AppBar dengan ikon yang berbeda.

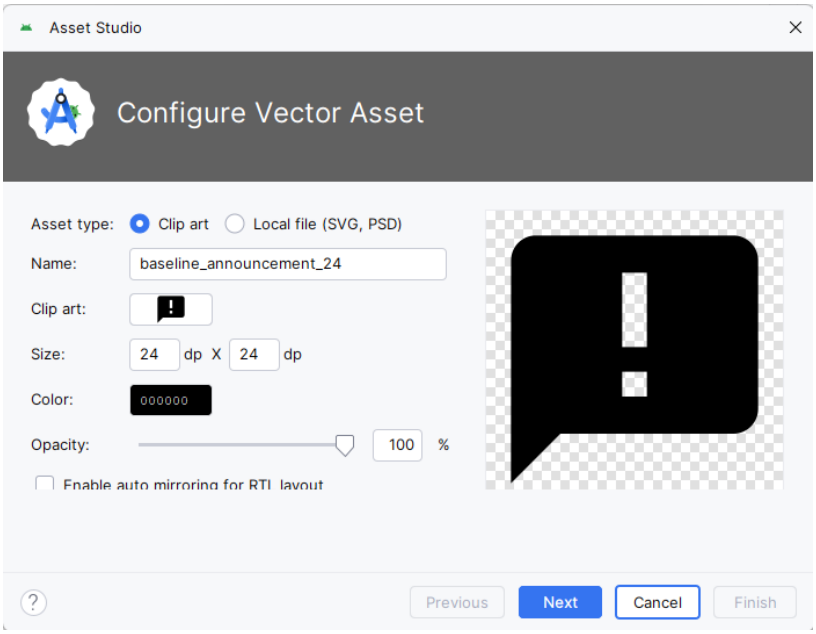
Terdapat 5 kondisi yang dapat dipasang pada tag app:showAsAction:

1. ifRoom, akan menampilkan action ketika ada ruangan pada AppBar.
2. withText, akan menampilkan actionitem beserta judulnya.
3. never, tidak akan pernah ditampilkan pada AppBar dan hanya akan ditampilkan pada overflow menu.
4. always, akan selalu tampil pada AppBar
5. collapseActionView, berhubungan dengan komponen collapsible.

Anda bisa mengunduh icon di tautan :

https://fonts.google.com/icons





atau dengan cara klik kanan pada folder drawable → New → Vector Asset → Pada Clipart cari "announcement" → Next → Finish.

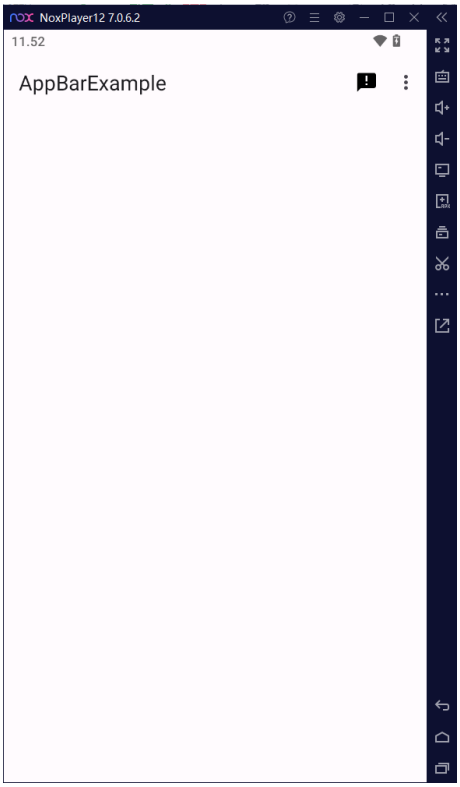
6. Selanjutnya, ubahlah ConstraintLayout menjadi CoordinatorLayout pada main_activity.xml dan tambahkan AppBarLayout beserta MaterialToolbar seperti berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/topAppBar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:menu="@menu/option_menu"
            app:title="@string/app_name" />
    </com.google.android.material.appbar.AppBarLayout>

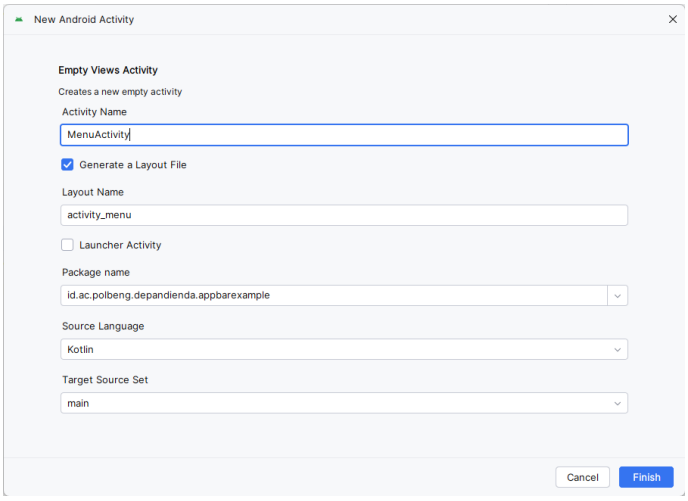
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

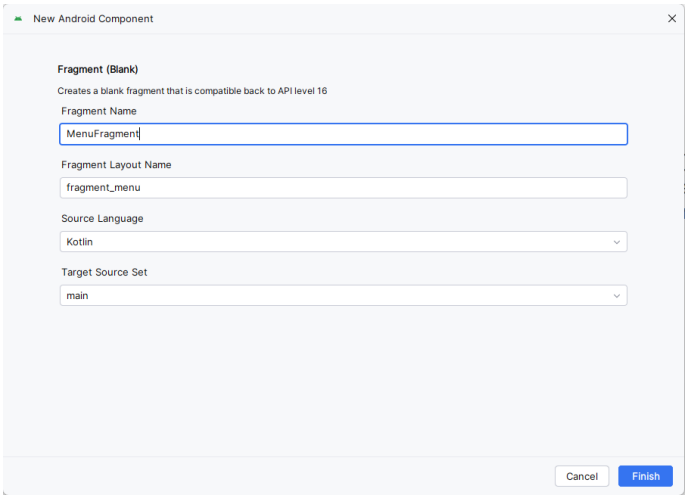
Sampai sini, cobalah jalankan aplikasi dan lihat hasilnya.

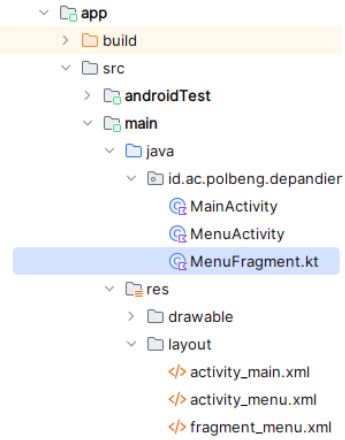


Sampai disini seharusnya dua buah menu sudah tampil. Nah, pada langkah selanjutnya, kita akan menambahkan aksi pada kedua menu tersebut. Menu 1 digunakan untuk menampilkan Fragment sedangkan Menu 2 digunakan untuk memanggil Activity.

7. Selanjutnya buatlah 1 Activity baru dan 1 Fragment baru dengan nama MenuActivity dan MenuFragment. Sehingga struktur project menjadi seperti ini.







8. Kemudian tambahkan sebuah TextView di activity_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MenuActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/this_activity"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

9. Begitu juga dengan layout dari MenuFragment. Tambahkan sebuah TextView sehingga isinya menjadi seperti berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MenuFragment">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_margin="16dp"
        android:text="@string/large_text" />

</FrameLayout>
```

10. Jangan lupa juga untuk menambahkan container pada activity_main.xml sebagai tempat menampilkan Fragment.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    ...
    tools:context=".MainActivity">

    <com.google.android.material.appbar.AppBarLayout
        ...
    </com.google.android.material.appbar.AppBarLayout>

    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior">

        <FrameLayout
            android:id="@+id/fragment_container"
            android:layout_width="match_parent"
            android:layout_height="match_parent" />
    </androidx.core.widget.NestedScrollView>

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Di sini Anda menggunakan beberapa komponen, berikut ini adalah fungsinya.

- CoordinatorLayout adalah sebuah ViewGroup yang dirancang untuk mengatur interaksi antara elemen-elemen tampilan (views) di dalamnya. Misalnya pada kasus ini, warna AppBar akan berubah ketika ada elemen di bawahnya yang di-scroll menggunakan trigger appbar_scrolling_view_behavior.
- AppBarLayout adalah sebuah ViewGroup yang dirancang untuk digunakan sebagai bagian atas dari tampilan aplikasi Anda.
- MaterialToolBar adalah komponen yang dapat digunakan untuk menampilkan judul, ikon untuk aksi, dan menu dalam aplikasi Anda.
- NestedScrollView adalah subclass dari ScrollView yang memungkinkan penggunaan tampilan yang dapat di-scroll ke dalam tampilan yang dapat di-scroll lainnya, misalnya RecyclerView di dalam ScrollView.

11. Setelah itu, gunakan fungsi fungsi setOnMenuItemClickListener pada MainActivity untuk menambahkan aksi seperti berikut:

```
package id.ac.polbeng.depandienda.appbarexample

import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import
id.ac.polbeng.depandienda.appbarexample.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

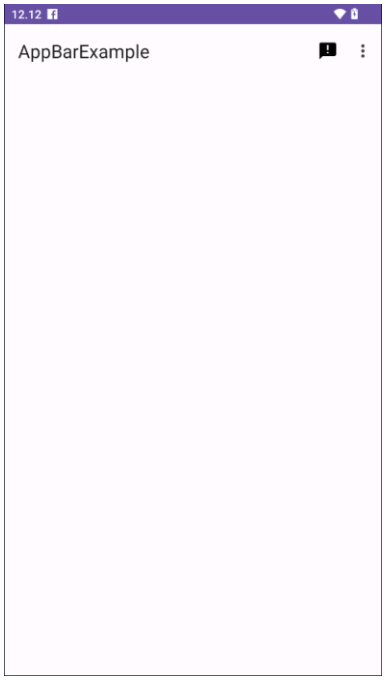
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

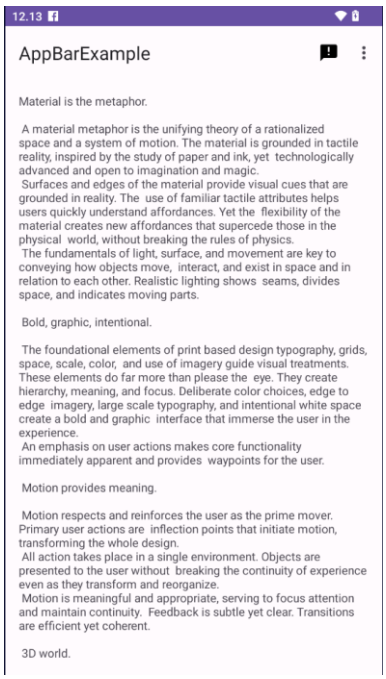
        binding.topAppBar.setOnMenuItemClickListener { menuItem ->
            when (menuItem.itemId) {
                R.id.menu1 -> {
                    supportFragmentManager.beginTransaction()
                        .replace(R.id.fragment_container, MenuFragment())
                        .addToBackStack(null)
                        .commit()
                    true
                }
                R.id.menu2 -> {
                    val intent = Intent(this, MenuActivity::class.java)
                    startActivity(intent)
                    true
                }
                else -> false
```

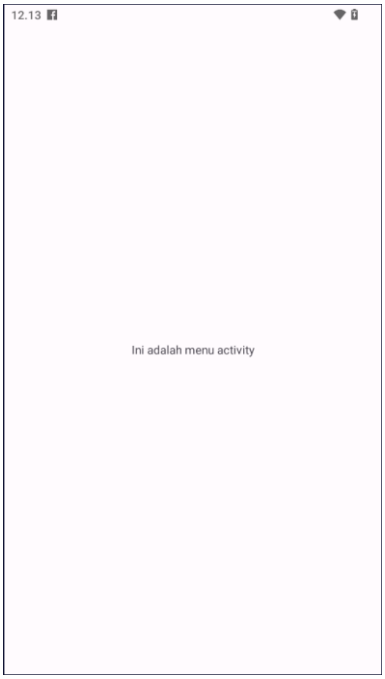
```
            }
        }
    }
}
```

Untuk mengetahui aksi ketika suatu aksi ditekan, kita menggunakan **setOnMenuItemClickListener** yang mengembalikan **menuItem**. Dengan melakukan percabangan menggunakan **switch** kita dapat memberikan listener pada setiap item. Seperti contoh di atas, ketika id-nya adalah R.id.menu1 maka yang terjadi adalah fragment pada R.id.fragment_container diganti dengan fragment baru yaitu **MenuFragment**. Ketika R.id.menu2 diklik, maka aplikasi akan menjalankan activity baru yaitu **MenuActivity**.

12. Setelah selesai silakan jalankan aplikasi. Tampilannya kurang lebih akan menjadi seperti gambar di bawah ini:







Latihan Mengimplementasikan SearchBar dan SearchView

Sebelumnya kita telah membuat AppBar untuk memanggil Activity baru melalui Menu 2. OK, kini kita lanjutkan untuk membuat menu pencarian menggunakan SearchBar dan SearchView pada Activity baru tersebut.

1. Pertama, tambahkan dulu resource string yang akan digunakan pada res → values → strings.xml seperti di bawah ini :

```
<resources>
    <string name="app_name">AppBarExample</string>
    ...
```

```
    <string name="searchbar_hint">Masukkan kata</string>
</resources>
```

2. Kemudian tambahkan SearchBar dan SearchView dari Material 3 pada activity_menu.xml seperti berikut.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MenuActivity">

    <com.google.android.material.search.SearchBar
        android:id="@+id/searchBar"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginHorizontal="16dp"
        android:layout_marginTop="16dp"
        android:hint="@string/searchbar_hint"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">
    </com.google.android.material.search.SearchBar>

    <com.google.android.material.search.SearchView
        android:id="@+id/searchView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:hint="@string/searchbar_hint"
        app:layout_anchor="@id/searchBar"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">
    </com.google.android.material.search.SearchView>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/this_activity"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

3. Nah, langkah selanjutnya yaitu menghubungkan kedua komponen tersebut pada MenuActivity. Sehingga, ketika SearchBar ditekan akan menampilkan SearchView.

```
package id.ac.polbeng.depandienda.appbarexample

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import
id.ac.polbeng.depandienda.appbarexample.databinding.ActivityMenuBinding

class MenuActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMenuBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMenuBinding.inflate(layoutInflater)
        setContentView(binding.root)

        with(binding) {
            searchView.setupWithSearchBar(searchBar)
        }
    }
}
```

4. Sampai sini, ketika SearchBar ditekan sudah dapat menampilkan SearchView tetapi tidak terjadi apapun ketika SearchView diisi dan ditekan Enter. Untuk menangani aksi tersebut, tambahkan listener berikut.

```
package id.ac.polbeng.depandienda.appbarexample

import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import
id.ac.polbeng.depandienda.appbarexample.databinding.ActivityMenuBinding

class MenuActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMenuBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMenuBinding.inflate(layoutInflater)
        setContentView(binding.root)
```

```
        with(binding) {
            searchView.setupWithSearchBar(searchBar)
            searchView
                .editText
                .setOnEditorActionListener { textView, actionId, event ->
                    searchBar.setText(searchView.text)
                    searchView.hide()
                    Toast.makeText(this@MenuActivity, searchView.text,
Toast.LENGTH_SHORT).show()
                    false
                }
        }
    }
}
```

Silakan jalankan aplikasinya:

